WSQ COURSE SLIDES

Create RESTful APIs and Web Apps with Python Flask

FastAPI · OpenAPI 3.1 · SQLite · VS Code · AI Vibe Coding

Course Code TGS-2021010365
Tertiary Infotech Academy Pte Ltd · UEN 201200696W

Version v11.0 · 5 September 2026

COURSE ADMINISTRATION

Digital Attendance (Mandatory)

1

AM attendance

Scan the SSG QR code at the start of each day.

2

PM attendance

Scan again after lunch.

3

Assessment attendance

A separate assessment attendance is required.

4

Eligibility

Maintain at least 75% attendance and complete assessments.

About the Trainer

Your Trainer

General trainer template — complete before class

NAME _____

DESIGNATION _____

QUALIFICATIONS _____

TECHNICAL EXPERTISE _____

INDUSTRY AND TRAINING EXPERIENCE _____

YOUR TRAINER

About the Trainer

Dr Alfred Ang

Principal Trainer · Tertiary Infotech Academy

ROLE	Founder and principal trainer
FOCUS	AI, cloud, software engineering and applied digital skills
QUALIFICATIONS	PhD · DACE · ACTA
TEACHING APPROACH	Mechanism, evidence, safe experimentation and verification

COURSE ADMINISTRATION

Let’s Know Each Other

1

Role

What systems or business processes do you work with?

2

Experience

What have you built with Python, JavaScript or APIs?

3

Goal

What API should you be able to build after this course?

4

Risk

What is one concern about AI-generated code?

Ground Rules

1. Participate actively and ask technical questions.
2. One conversation at a time; respect different approaches.
3. Use only the authorised local demo data and endpoints.
4. Never paste live credentials or personal data into AI tools.
5. Return from breaks on time.
6. Capture evidence as you complete each lab.

COURSE ADMINISTRATION

Course at a Glance

1

Course

Create RESTful APIs and Web Apps with Python Flask

2

Code

TGS-2021010365

3

Duration

2 days · 16 training hours

4

Level

Beginner

5

TSC

Applications Integration · ICT-DIT-3003-1.1

6

Stack

OpenAI Python SDK · FastAPI · SQLite · browser client

Learning Outcomes

1

LO1

Identify and assess FastAPI as middleware for creating connections between web clients, applications and data stores.

2

LO2

Integrate data and functions into a browser web app through typed FastAPI routes and reusable Python services.

3

LO3

Support API-level integration using REST semantics, JSON and an OpenAPI 3.1 contract.

4

LO4

Perform tests and checks on FastAPI-to-SQLite connections using Swagger UI, pytest, logs and VS Code debugging.

5

LO5

Modify a FastAPI service to improve integration, validation, compatibility, performance and security.

TSC ALIGNMENT

Knowledge Criteria K1–K5

1

K1

Types of middleware and their features

2

K2

Proper usage of middleware

3

K3

Different types of platforms on which applications run

4

K4

Potential technical, compatibility or performance issues in application integration

5

K5

Functions of Application Programming Interfaces and OpenAPI descriptions

TSC ALIGNMENT

Ability Criteria A1–A4

1

A1

Identify opportunities for creating connections among devices, databases, software and applications

2

A2

Perform a feasibility scan to identify suitable middleware

3

A3

Use middleware to integrate data and functions across application programs

4

A4

Support API-level integration

TSC ALIGNMENT

Ability Criteria A5–A8

1

A5

Perform tests and checks on connections between disparate application programs

2

A6

Verify proper functioning across integrated platforms

3

A7

Highlight technical, compatibility or performance issues following integration

4

A8

Modify middleware or programming processes to enhance integration and connections

COURSE STRUCTURE

Six-Topic Technical Map

1

Topic 01

REST API Fundamentals and AI Vibe Coding

2

Topic 02

AI-Assisted API Functions, Routing and Security

3

Topic 03

Request Handling, Data Validation and JSON Responses

4

Topic 04

API Error Handling, Testing and Debugging

5

Topic 05

API Architecture, Data Models and Database Integration

6

Topic 06

REST API Documentation, Integration and Deployment

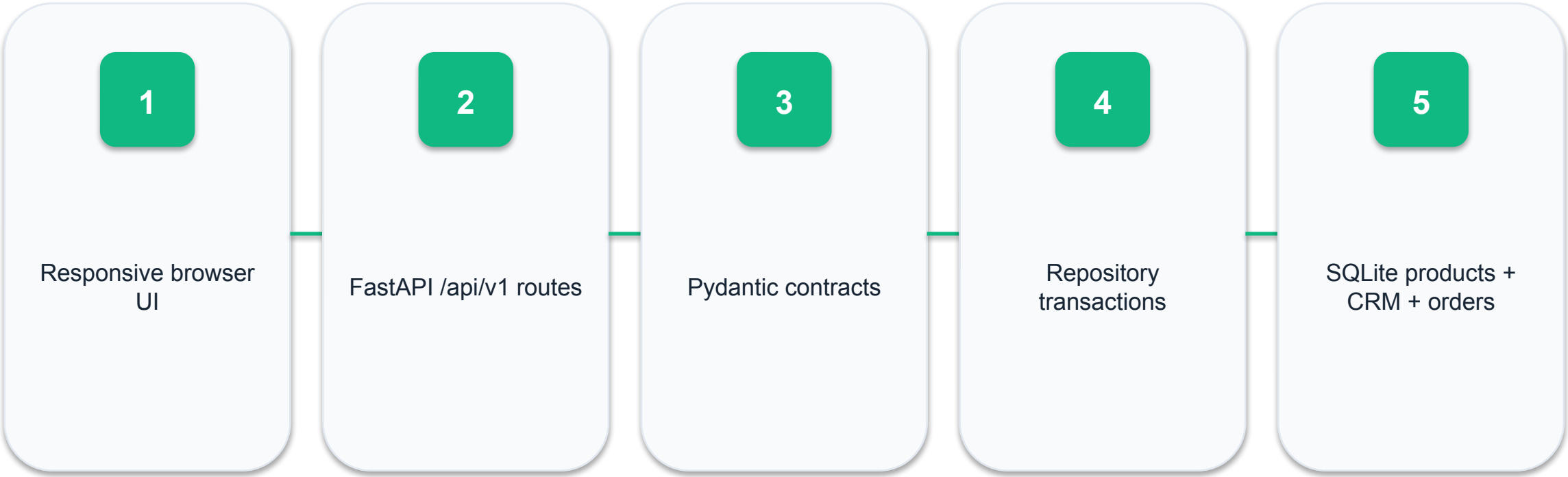
DEVELOPMENT ENVIRONMENT

VS Code Engineering Workspace

1. Python + Pylance
2. REST Client
3. SQLite Viewer
4. YAML validation
5. pytest + debugger

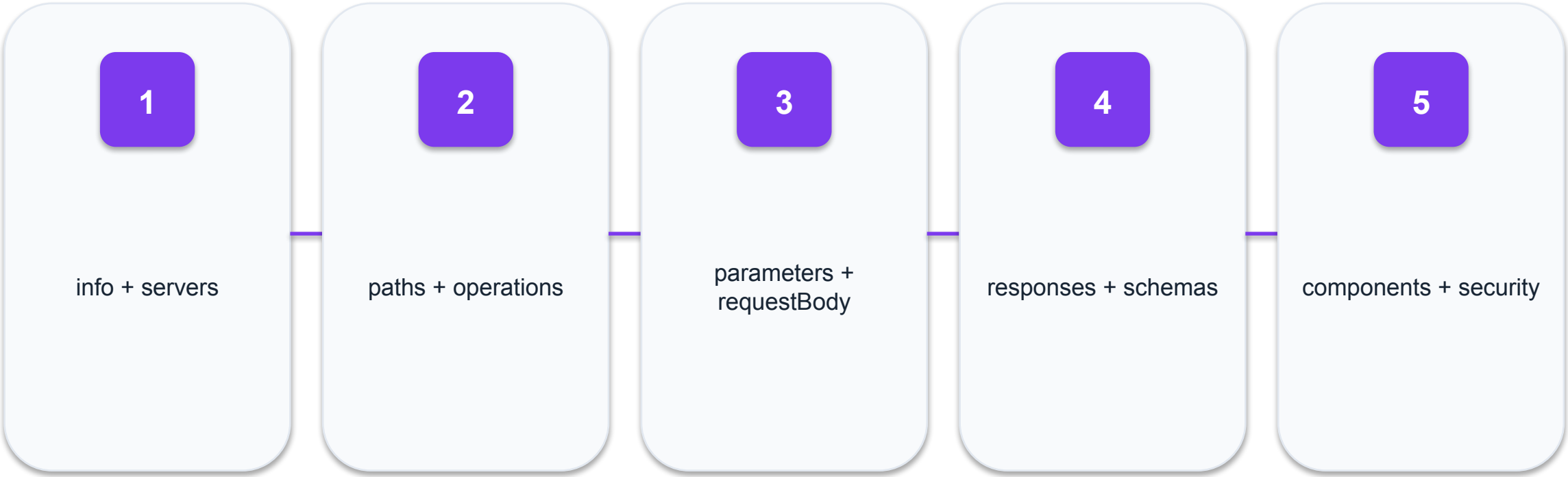
REALISTIC CAPSTONE

Northstar Commerce Architecture

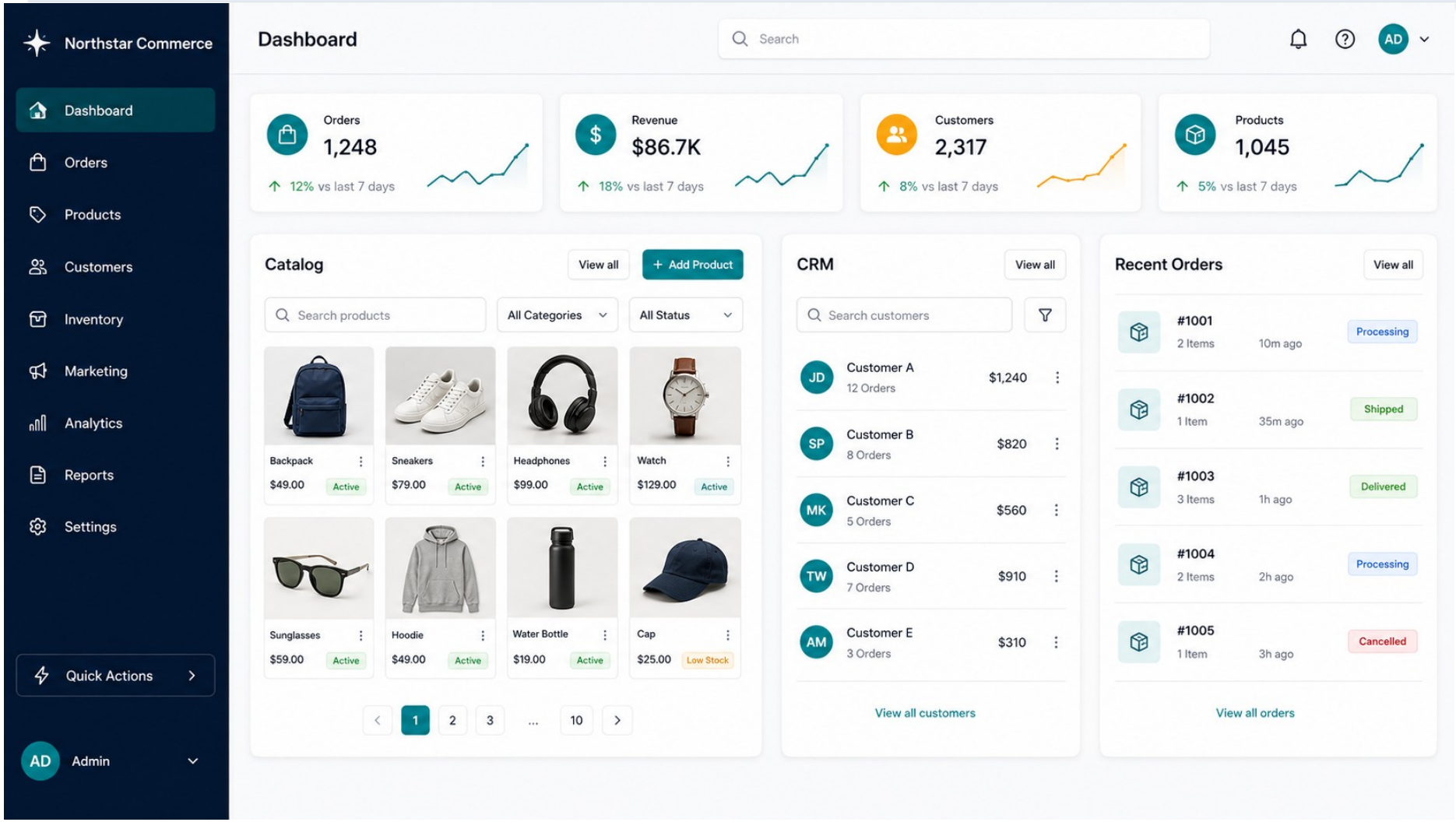


CONTRACT FIRST

OpenAPI 3.1 Contract Map



Northstar Commerce operations dashboard



CONCEPT VISUAL

Products

Customers

Orders

Revenue

Runnable UI appears in Lab 11.

Source: Imagegen concept; not measured data

LESSON PLAN

Day 1 · Contract to Validation

1

09:00

09:30 · Administration, outcomes and course contract

2

09:30

11:00 · Topic 1 concepts and demonstrations

3

11:00

12:30 · Labs 1-2

4

12:30

13:30 · Lunch

5

13:30

15:00 · Topic 2 concepts and demonstrations

6

15:00

16:15 · Labs 3-4

7

16:15

17:15 · Topic 3 concepts and demonstrations

8

17:15

18:00 · Labs 5-6 briefing and guided start

LESSON PLAN

Day 2 · Data to Deployment

1

09:00

10:30 · Topic 4 concepts and demonstrations

2

10:30

12:00 · Labs 7-8

3

12:00

13:00 · Topic 5 architecture and SQLite

4

13:00

14:00 · Lunch

5

14:00

15:30 · Labs 9-10

6

15:30

16:30 · Topic 6 integration and deployment

7

16:30

17:45 · Labs 11-12 capstone

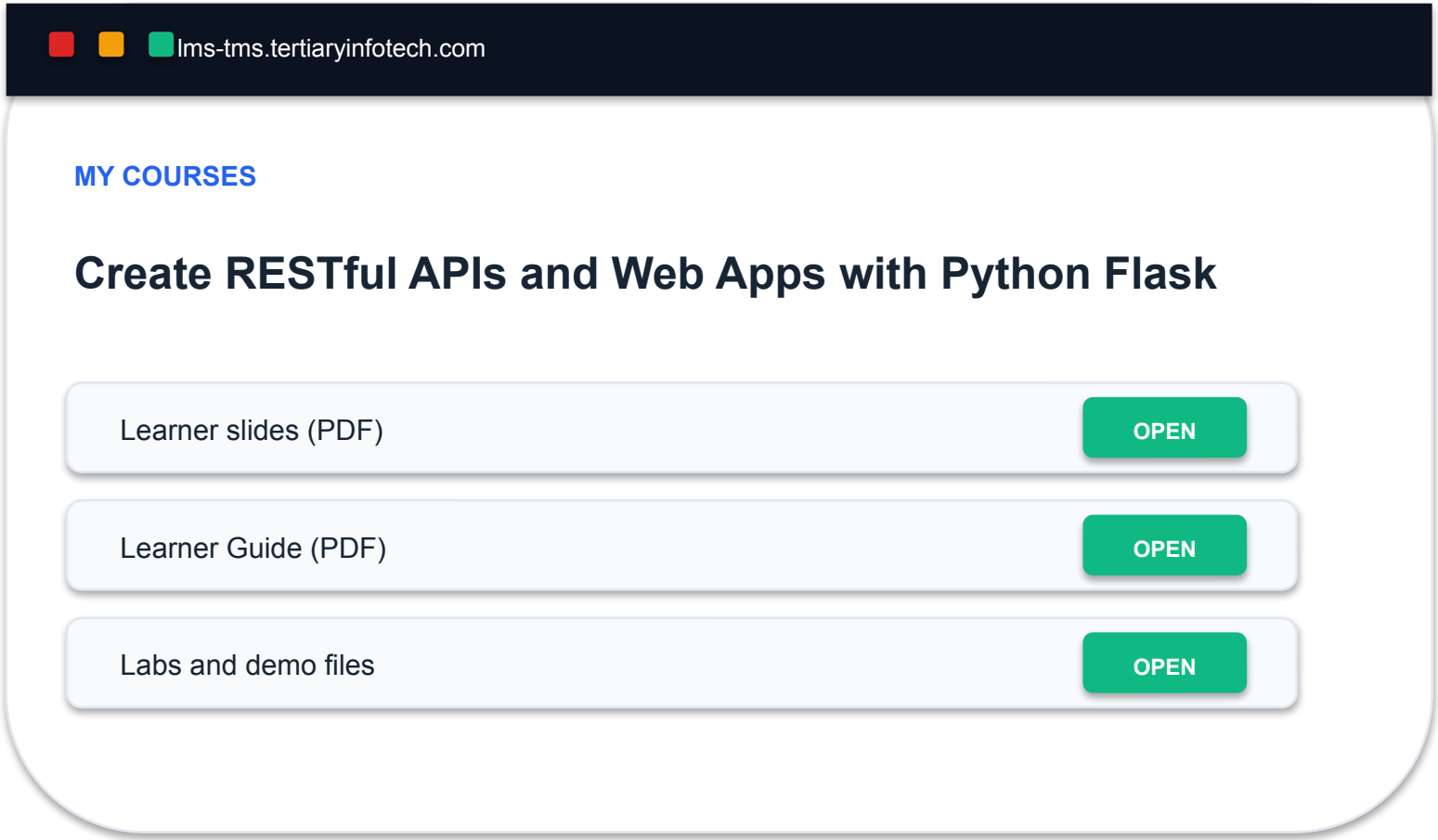
8

17:45

18:00 · Review, TRAQOM and assessment briefing

COURSE MATERIAL

Download Course Material



Learner evidence

Confirm the current version, then download the PDF guide and Labs folder before class starts.

BEFORE ASSESSMENT

Assessment Briefing

1

Format

Open book; individual work only.

2

WA

10 open-ended questions · 60 minutes.

3

PP

6 practical tasks · 90 minutes.

4

Materials

Slides, Learner Guide and approved lab files.

5

Conduct

No discussion, photos or recording of scripts.

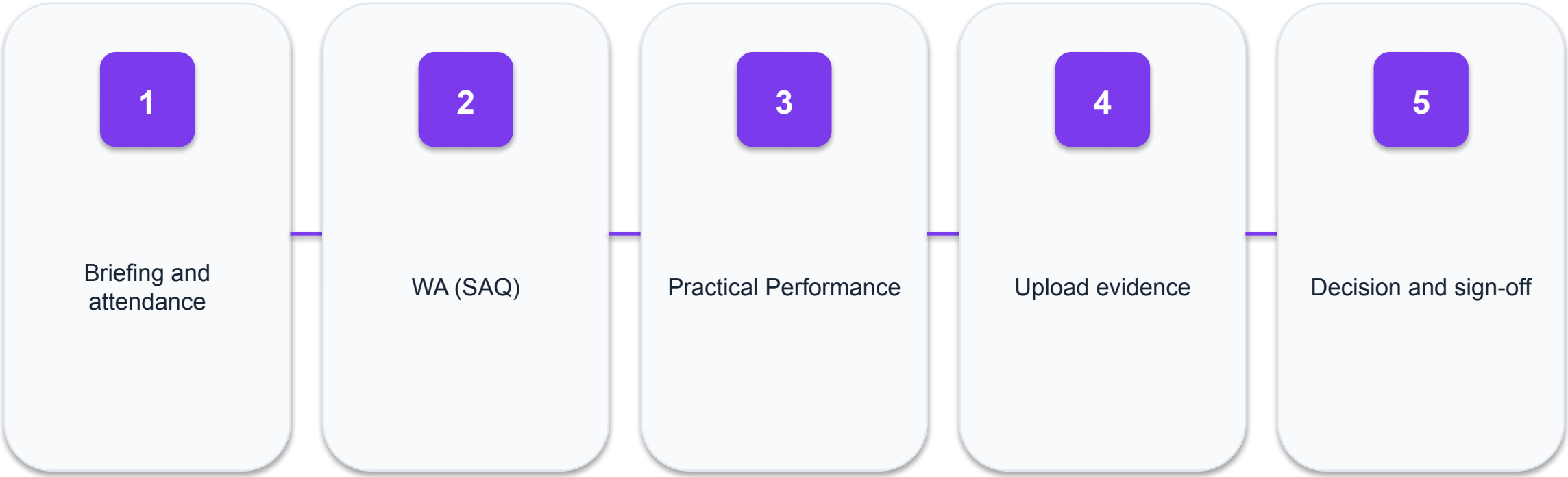
6

Submission

Use the LMS-TMS upload route named in the paper.

ASSESSMENT

Assessment Flow



TOPIC 01

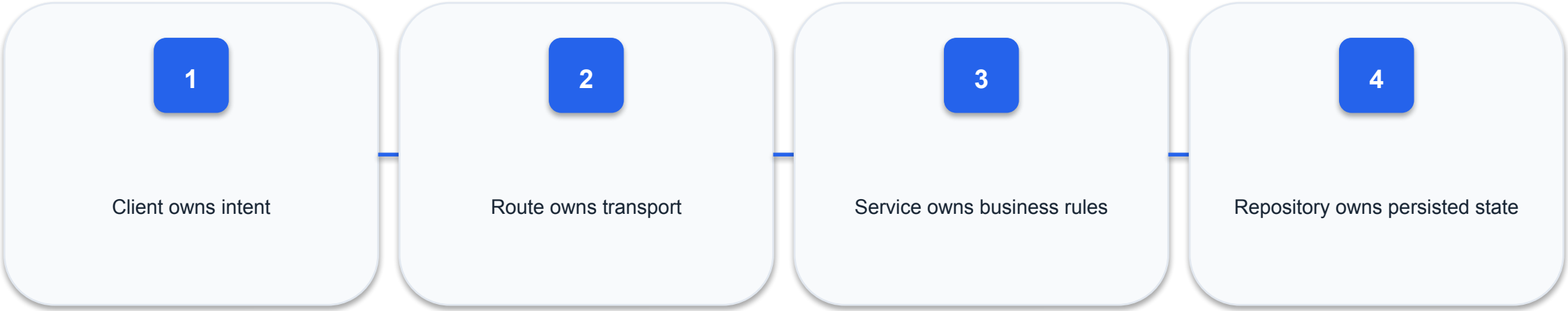
REST API Fundamentals and AI Vibe Coding

Contracts · resources · HTTP semantics · OpenAI Python SDK · prompt-to-test workflow

01

The API contract is the product boundary

A request crosses a boundary; the contract makes method, path, data and outcomes explicit.



ARTIFACT GET /api/v1/products?status=open → 200 + JSON array

FAILURE A prompt that says only ‘build an API’ leaves every contract decision implicit.

VERIFY A reviewer can predict the request and response without reading implementation code.

Prove: The API contract is the product boundary

INPUT

Client owns intent

MECHANISM

A request crosses a boundary; the contract makes method, path, data and outcomes explicit.

FAILURE

A prompt that says only ‘build an API’ leaves every contract decision implicit.

PROOF

A reviewer can predict the request and response without reading implementation code.

Evidence artifact: GET /api/v1/products?status=open → 200 + JSON array

Model resources before routes

Use nouns for resources and HTTP methods for operations.

1. Collection: /api/v1/products
2. Member: /api/v1/products/{product_id}
3. Filter: ?status=open
4. Avoid verbs such as /getProducts

ARTIFACT Resource map: Product {id,sku,name,price,stock_quantity,status}

FAILURE Action-style URLs multiply and weaken predictable semantics.

VERIFY Every route maps to one resource state transition.

Prove: Model resources before routes

INPUT

Collection: /api/v1/products

MECHANISM

Use nouns for resources and HTTP methods for operations.

FAILURE

Action-style URLs multiply and weaken predictable semantics.

PROOF

Every route maps to one resource state transition.

Evidence artifact: Resource map: Product {id,sku,name,price,stock_quantity,status}

HTTP methods encode intent

GET reads, POST creates, PUT replaces, PATCH changes part, DELETE removes.

1. GET is safe
2. PUT is idempotent
3. POST is normally non-idempotent
4. PATCH needs an explicit partial-update rule

ARTIFACT	Method × state-change matrix
FAILURE	Using POST for every action hides semantics from clients and tools.
VERIFY	Repeat an idempotent request and confirm the resulting state is unchanged.

Prove: HTTP methods encode intent

INPUT

GET is safe

MECHANISM

GET reads, POST creates, PUT replaces, PATCH changes part, DELETE removes.

FAILURE

Using POST for every action hides semantics from clients and tools.

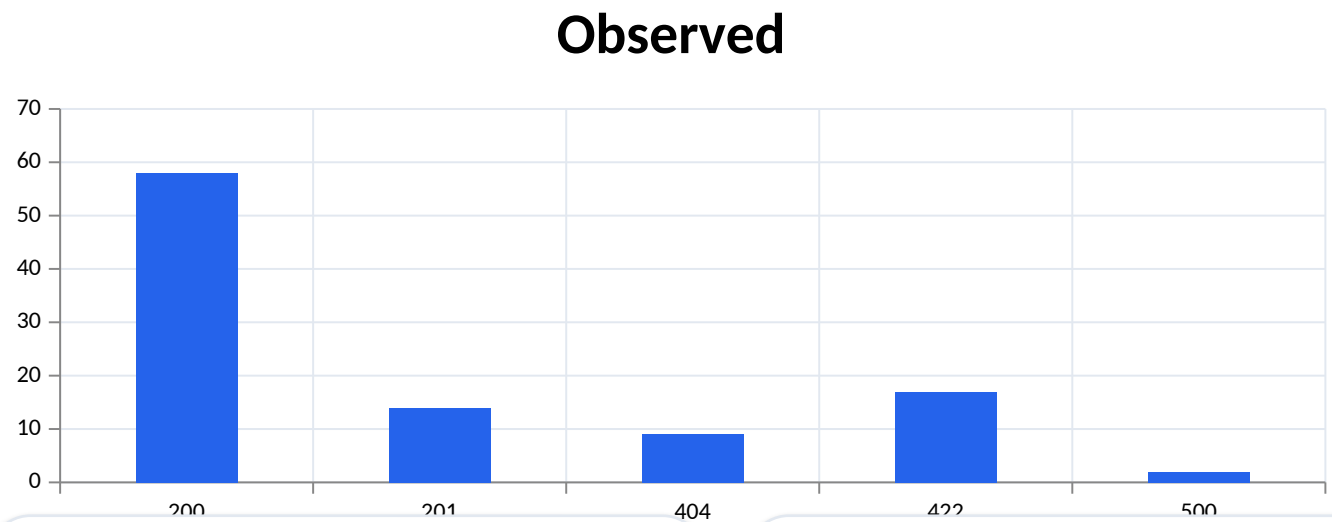
PROOF

Repeat an idempotent request and confirm the resulting state is unchanged.

Evidence artifact: Method × state-change matrix

Status codes are machine-readable outcomes

Status codes separate transport outcome from response detail.



01 2xx success

02 4xx client correction

03 5xx server investigation

04 Body carries actionable detail

ARTIFACT Response distribution from a 100-request test run

FAILURE Returning 200 for failures breaks monitoring and client control flow.

VERIFY Success, validation, missing-resource and conflict paths each assert a specific code.

Prove: Status codes are machine-readable outcomes

INPUT

2xx success

MECHANISM

Status codes separate transport outcome from response detail.

FAILURE

Returning 200 for failures breaks monitoring and client control flow.

PROOF

Success, validation, missing-resource and conflict paths each assert a specific code.

Evidence artifact: Response distribution from a 100-request test run

FastAPI turns types into runtime contracts

Python annotations drive parsing, validation, editor help and OpenAPI schemas.

```
@app.get("/api/v1/products/{product_id}")
def read_product(product_id: int) -> ProductRead:
    return service.get(product_id)
```

- Path values are converted
- Pydantic validates bodies
- Return types filter responses
- OpenAPI is generated

ARTIFACT Typed route → validation → schema

FAILURE Untyped dictionaries defer errors until later code paths.

VERIFY Send an invalid type and inspect the structured 422 error location.

Prove: FastAPI turns types into runtime contracts

INPUT

Path values are converted

MECHANISM

Python annotations drive parsing, validation, editor help and OpenAPI schemas.

FAILURE

Untyped dictionaries defer errors until later code paths.

PROOF

Send an invalid type and inspect the structured 422 error location.

Evidence artifact: Typed route → validation → schema

The ASGI request lifecycle

Uvicorn accepts the connection, FastAPI resolves the route and dependencies, then serializes the response.

ARTIFACT Browser → Uvicorn → FastAPI → service → JSON

FAILURE Blocking work in an async handler can stall concurrent requests.

VERIFY Trace one request using a request ID across access and application logs.

Prove: The ASGI request lifecycle

INPUT

socket

MECHANISM

Uvicorn accepts the connection, FastAPI resolves the route and dependencies, then serializes the response.

FAILURE

Blocking work in an async handler can stall concurrent requests.

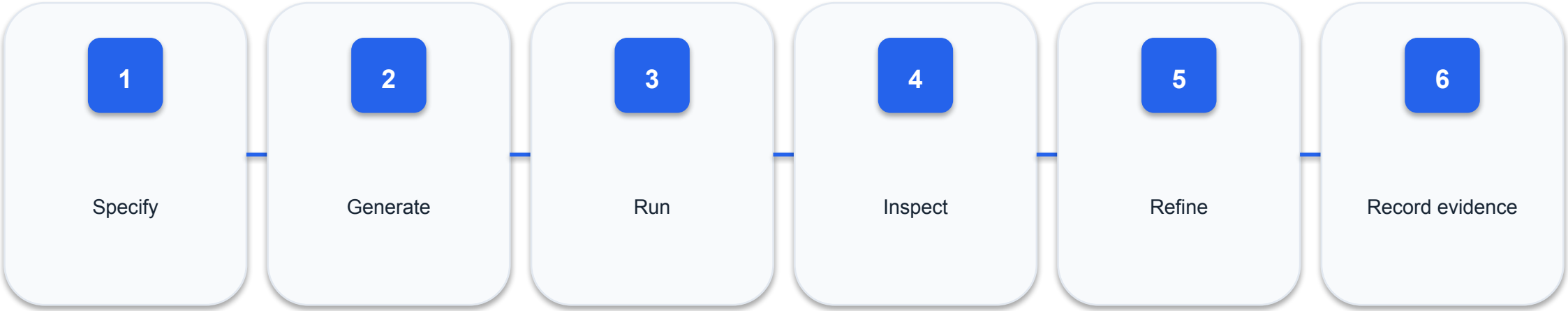
PROOF

Trace one request using a request ID across access and application logs.

Evidence artifact: Browser → Uvicorn → FastAPI → service → JSON

Vibe coding needs a closed verification loop

Prompting accelerates drafting; tests and review decide whether generated code is acceptable.



ARTIFACT Prompt + acceptance checks + diff

FAILURE Accepting plausible code without executing it transfers uncertainty into production.

VERIFY Every prompt names inputs, outputs, constraints and tests; every change produces evidence.

Source: VS Code FastAPI Tutorial

Prove: Vibe coding needs a closed verification loop

INPUT

Specify

MECHANISM

Prompting accelerates drafting; tests and review decide whether generated code is acceptable.

FAILURE

Accepting plausible code without executing it transfers uncertainty into production.

PROOF

Every prompt names inputs, outputs, constraints and tests; every change produces evidence.

Evidence artifact: Prompt + acceptance checks + diff

A good prompt is an executable design brief

Constrain framework, paths, schemas, errors, persistence and tests before asking AI to code.

1 Weak: build a product API

2 Strong: FastAPI, /api/v1/products, SQLite, validation, 404/409, pytest, no ORM

ARTIFACT Prompt contract checklist

FAILURE Over-specific implementation prompts can freeze a poor design; specify behaviour first.

VERIFY Generated endpoints match a written route table and pass named tests.

Prove: A good prompt is an executable design brief

INPUT

Weak: build a product API

MECHANISM

Constrain framework, paths, schemas, errors, persistence and tests before asking AI to code.

FAILURE

Over-specific implementation prompts can freeze a poor design; specify behaviour first.

PROOF

Generated endpoints match a written route table and pass named tests.

Evidence artifact: Prompt contract checklist

The Python SDK turns a prompt into a typed file bundle

A reproducible vibe-coding run records the model, prompt, mock context and structured output before any code is accepted.

```
client = OpenAI()
rsp = client.responses.parse(
    model=os.environ["OPENAI_MODEL"],
    input=messages, text_format=CodeBundle
)
bundle = read_bundle(rsp)
```

- OpenAI() reads OPENAI_API_KEY
- responses.parse sends the brief
- CodeBundle constrains file paths and contents
- generated/ keeps AI output separate for review

ARTIFACT ai_vibe.py + Prompts.pdf + mock-data.json

FAILURE Copying unstructured model text directly over working source hides provenance and can overwrite trusted code.

VERIFY The SDK produces a validated CodeBundle in generated/; pytest and human review decide whether to apply it.

Prove: The Python SDK turns a prompt into a typed file bundle

INPUT

OpenAI() reads OPENAI_API_KEY

MECHANISM

A reproducible vibe-coding run records the model, prompt, mock context and structured output before any code is accepted.

FAILURE

Copying unstructured model text directly over working source hides provenance and can overwrite trusted code.

PROOF

The SDK produces a validated CodeBundle in generated/; pytest and human review decide whether to apply it.

Evidence artifact: ai_vibe.py + Prompts.pdf + mock-data.json

LAB 01 · SCENARIO

Prompt a REST API Contract

DESIGN CHALLENGE

Convert Northstar Commerce requirements into a versioned OpenAPI-first route and data contract.

Build: api-contract.md and openapi-draft.yaml

TOOLS

Browser, VS Code, YAML extension, AI coding assistant, OpenAI Python SDK

Detailed procedure: Learner Guide + lab README

Prompt submitted by ai_vibe.py

LEARNER PROMPT

```
You are working on Lab 01 of a Northstar Commerce API. Create a
behaviour-first OpenAPI 3.1 contract for Northstar Commerce using
FastAPI and SQLite. Define Product, Customer, Order and OrderItem
schemas; list collection/member routes, request and response JSON,
200/201/204/401/404/409/422 outcomes, and at least six acceptance
tests. Do not write implementation code yet. Return only api-
contract.md, openapi-draft.yaml and machine-readable test cases.
```

PYTHON SDK ENVELOPE

- Prompts.pdf
- mock-data.json
- **responses.parse**
- **CodeBundle**
- generated/
- pytest + review

Full prompt + refinement
in the lab Prompts.pdf

How Prompt a REST API Contract works

1

Input contract

Convert Northstar Commerce requirements into a versioned OpenAPI-first route and data contract.

2

Boundary validation

Reject invalid or unauthorised input

3

Business decision

Apply one explicit rule

4

State/evidence

The contract covers products, customers and orders, names 200/201/204/401/404/409/422 outcomes and validates as OpenAPI 3.1.

LAB 01 · EVIDENCE

api-contract.md and openapi-draft.yaml

```
REQUEST
  lab: 01
  objective: Convert Northstar Commerce requirements into a
             versioned OpenAPI-first route and data
             contract.

EXPECTED EVIDENCE
  artifact: api-contract.md and openapi-draft.yaml
  codes: A1, A2
  status: verified
```

ASSESSMENT BRIDGE

Ability codes
A1, A2

Capture a screenshot, terminal output or test result that proves the acceptance criterion.

LAB 01 · VERIFY

Acceptance and diagnosis

PASS WHEN

The contract covers products, customers and orders, names 200/201/204/401/404/409/422 outcomes and validates as OpenAPI 3.1.

DIAGNOSE

1. Inspect HTTP status and response body
2. Check request fields and headers
3. Follow request ID into logs
4. Re-run the smallest failing test

Prompt output must preserve the API boundary

TARGET FILES

`api-contract.md`

`tests/contract-cases.json`

ACCEPTANCE

The contract covers products, customers and orders, names 200/201/204/401/404/409/422 outcomes and validates as OpenAPI 3.1.

ABILITY TRACE

A1, A2

LAB 02 · SCENARIO

Run the First FastAPI Service

DESIGN CHALLENGE

Create and verify a typed product endpoint and its generated OpenAPI documentation.

Build: main.py with /health and /api/v1/products routes

TOOLS

Python, FastAPI, Uvicorn, Swagger UI, OpenAI Python SDK

Detailed procedure: Learner Guide + lab README

Prompt submitted by ai_vibe.py

LEARNER PROMPT

```
You are working on Lab 02 of a Northstar Commerce API. Generate
the smallest typed FastAPI service with GET /health, GET
/api/v1/products and GET /api/v1/products/{product_id}. Use
Pydantic response models, integer path validation and OpenAPI
summaries. Add TestClient tests for health, route inventory and a
non-integer product id returning structured 422 JSON.
```

PYTHON SDK ENVELOPE

- Prompts.pdf
- mock-data.json
- **responses.parse**
- **CodeBundle**
- generated/
- pytest + review

Full prompt + refinement
in the lab Prompts.pdf

Source: OpenAI Python SDK

LAB 02 · MECHANISM

How Run the First FastAPI Service works

1

Input contract

Create and verify a typed product endpoint and its generated OpenAPI documentation.

2

Boundary validation

Reject invalid or unauthorised input

3

Business decision

Apply one explicit rule

4

State/evidence

GET /health/live returns 200; /docs lists the product route; invalid path input returns structured 422 JSON.

LAB 02 · EVIDENCE

main.py with /health and /api/v1/products routes

```
REQUEST
  lab: 02
  objective: Create and verify a typed product endpoint and its
             generated OpenAPI documentation.

EXPECTED EVIDENCE
  artifact: main.py with /health and /api/v1/products routes
  codes: A1, A4
  status: verified
```

ASSESSMENT BRIDGE

Ability codes
A1, A4

Capture a screenshot, terminal output or test result that proves the acceptance criterion.

LAB 02 · VERIFY

Acceptance and diagnosis

PASS WHEN

GET /health/live returns 200; /docs lists the product route; invalid path input returns structured 422 JSON.

DIAGNOSE

1. Inspect HTTP status and response body
2. Check request fields and headers
3. Follow request ID into logs
4. Re-run the smallest failing test

LAB 02 · CONTRACT REVIEW

Prompt output must preserve the API boundary

TARGET FILES

`app/main.py`

`tests/test_health.py`

ACCEPTANCE

GET /health/live returns 200; /docs lists the product route; invalid path input returns structured 422 JSON.

ABILITY TRACE

A1, A4

TOPIC 02

AI-Assisted API Functions, Routing and Security

Routers · dependency injection · reusable services · CORS · API keys

02

Separate transport from business logic

Thin route functions translate HTTP; service functions express rules independently of the web framework.

1. Router: parse/return
2. Service: decisions
3. Repository: SQL
4. Schema: validate/serialize

ARTIFACT router → service → repository

FAILURE SQL embedded in every route creates duplication and fragile tests.

VERIFY The service can be unit-tested without starting Uvicorn.

Prove: Separate transport from business logic

INPUT

Router: parse/return

MECHANISM

Thin route functions translate HTTP; service functions express rules independently of the web framework.

FAILURE

SQL embedded in every route creates duplication and fragile tests.

PROOF

The service can be unit-tested without starting Uvicorn.

Evidence artifact: router → service → repository

APIRouter creates bounded route modules

A router groups paths, tags and dependencies under a shared prefix.

```
router = APIRouter(prefix="/api/v1/products",
tags=["products"])
app.include_router(router)
```

- prefix=/api/v1/products
- tags=[products]
- shared dependencies
- included once by app

ARTIFACT products.router mounted by main.py

FAILURE Duplicate prefixes and inconsistent tags produce confusing OpenAPI output.

VERIFY OpenAPI lists every product endpoint once under the intended tag.

Prove: APIRouter creates bounded route modules

INPUT

prefix=/api/v1/products

MECHANISM

A router groups paths, tags and dependencies under a shared prefix.

FAILURE

Duplicate prefixes and inconsistent tags produce confusing OpenAPI output.

PROOF

OpenAPI lists every product endpoint once under the intended tag.

Evidence artifact: products.router mounted by main.py

Dependency injection makes controls reusable

Depends() resolves shared behaviour before the handler and passes the verified result in.

ARTIFACT Depends(require_api_key)

FAILURE Copy-pasted authentication diverges route by route.

VERIFY Protected routes reject missing/invalid keys and accept the configured key.

Prove: Dependency injection makes controls reusable

INPUT

read header

MECHANISM

Depends() resolves shared behaviour before the handler and passes the verified result in.

FAILURE

Copy-pasted authentication diverges route by route.

PROOF

Protected routes reject missing/invalid keys and accept the configured key.

Evidence artifact: Depends(require_api_key)

API-key authentication is a boundary control

A demo API key illustrates authentication, but production needs rotation, secure storage and least privilege.

```
def require_api_key(x_api_key: str = Header()):
    if not secrets.compare_digest(x_api_key,
settings.api_key):
        raise HTTPException(401, "Invalid API key")
```

- X-API-Key header
- constant-time comparison
- environment configuration
- 401/403 policy

ARTIFACT Header → dependency → authorised principal

FAILURE Hard-coded secrets in source or frontend are publicly exposed.

VERIFY Secret scan finds no live key; unauthorised requests fail before business logic.

Prove: API-key authentication is a boundary control

INPUT

X-API-Key header

MECHANISM

A demo API key illustrates authentication, but production needs rotation, secure storage and least privilege.

FAILURE

Hard-coded secrets in source or frontend are publicly exposed.

PROOF

Secret scan finds no live key; unauthorised requests fail before business logic.

Evidence artifact: Header → dependency → authorised principal

CORS is a browser origin policy

Origin is scheme + host + port; the browser checks whether frontend JavaScript may read the response.

1. http://localhost:5500
2. http://localhost:8000
3. different ports = different origins
4. allow only required origins

ARTIFACT Preflight OPTIONS → CORS headers

FAILURE Wildcard origins cannot safely support credentials and widen exposure.

VERIFY Browser devtools shows the allowed origin and successful preflight.

Prove: CORS is a browser origin policy

INPUT

http://localhost:5500

MECHANISM

Origin is scheme + host + port; the browser checks whether frontend JavaScript may read the response.

FAILURE

Wildcard origins cannot safely support credentials and widen exposure.

PROOF

Browser devtools shows the allowed origin and successful preflight.

Evidence artifact: Preflight OPTIONS → CORS headers

Security requirements are testable behaviour

Translate ‘secure the endpoint’ into explicit threat, control and verification rows.

1. Threat: unauthorized write
2. Control: API key dependency
3. Failure: 401
4. Evidence: test + log

ARTIFACT Threat-control-test matrix

FAILURE A control with no negative test is an unverified assumption.

VERIFY Each security requirement has at least one rejection test.

Prove: Security requirements are testable behaviour

INPUT

Threat: unauthorized write

MECHANISM

Translate ‘secure the endpoint’ into explicit threat, control and verification rows.

FAILURE

A control with no negative test is an unverified assumption.

PROOF

Each security requirement has at least one rejection test.

Evidence artifact: Threat-control-test matrix

Review AI-generated security code adversarially

Generated code is a draft; inspect secrets, validation, SQL, error detail, CORS and dependency versions.

1. Plausible: hard-coded key
2. Acceptable: environment key + ignored .env
3. Plausible: f-string SQL
4. Acceptable: parameters

ARTIFACT Security review diff

FAILURE AI may optimise for a working demo rather than operational safety.

VERIFY A checklist review produces an approved diff and recorded exceptions.

Prove: Review AI-generated security code adversarially

INPUT

Plausible: hard-coded key

MECHANISM

Generated code is a draft; inspect secrets, validation, SQL, error detail, CORS and dependency versions.

FAILURE

AI may optimise for a working demo rather than operational safety.

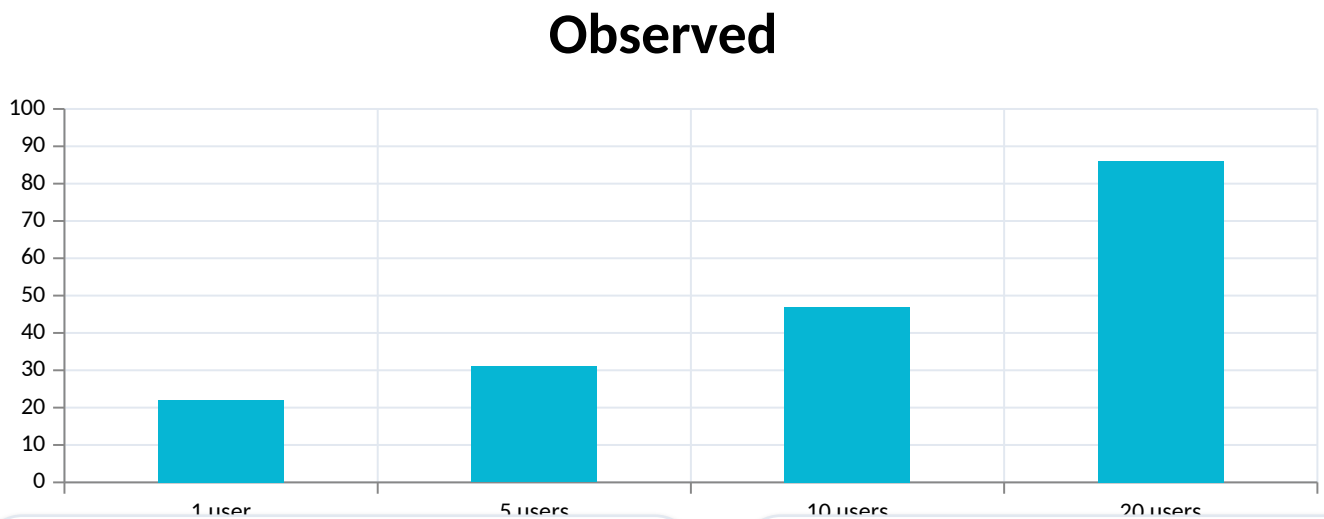
PROOF

A checklist review produces an approved diff and recorded exceptions.

Evidence artifact: Security review diff

Sync and async must match the work

Async improves concurrency for awaitable I/O; blocking SQLite calls belong in normal def handlers or a threadpool.



01 async def for awaitable clients

02 def for blocking libraries

03 measure, do not cargo-cult

04 keep transactions short

ARTIFACT Latency under 1 vs 20 concurrent requests

FAILURE Calling blocking work directly inside async def stalls the event loop.

VERIFY A concurrency test keeps p95 latency within the chosen threshold.

Prove: Sync and async must match the work

INPUT

async def for awaitable clients

MECHANISM

Async improves concurrency for awaitable I/O; blocking SQLite calls belong in normal def handlers or a threadpool.

FAILURE

Calling blocking work directly inside async def stalls the event loop.

PROOF

A concurrency test keeps p95 latency within the chosen threshold.

Evidence artifact: Latency under 1 vs 20 concurrent requests

LAB 03 · SCENARIO

Split Routes and Reuse a Service Function

DESIGN CHALLENGE

Refactor a monolithic FastAPI file into router and service modules without changing the contract.

Build: main.py, routes.py and service.py

TOOLS

FastAPI APIRouter, pytest, OpenAI Python SDK

Detailed procedure: Learner Guide + lab README

Prompt submitted by ai_vibe.py

LEARNER PROMPT

```
You are working on Lab 03 of a Northstar Commerce API. Refactor
the supplied FastAPI product service into thin routes and reusable
service functions. Mount one APIRouter at /api/v1/products,
preserve the existing contract and add regression tests proving
every OpenAPI operation appears once. Explain module ownership in
concise docstrings.
```

PYTHON SDK ENVELOPE

- Prompts.pdf
- mock-data.json
- **responses.parse**
- **CodeBundle**
- generated/
- pytest + review

Full prompt + refinement
in the lab Prompts.pdf

LAB 03 · MECHANISM

How Split Routes and Reuse a Service Function works

1

Input contract

Refactor a monolithic FastAPI file into router and service modules without changing the contract.

2

Boundary validation

Reject invalid or unauthorised input

3

Business decision

Apply one explicit rule

4

State/evidence

All baseline tests still pass and OpenAPI exposes each route exactly once.

main.py, routes.py and service.py

```
REQUEST
  lab: 03
  objective: Refactor a monolithic FastAPI file into router and
             service modules without changing the
             contract.

EXPECTED EVIDENCE
  artifact: main.py, routes.py and service.py
  codes: A3, A8
  status: verified
```

ASSESSMENT BRIDGE

Ability codes
A3, A8

Capture a screenshot, terminal output or test result that proves the acceptance criterion.

Acceptance and diagnosis

PASS WHEN

All baseline tests still pass and OpenAPI exposes each route exactly once.

DIAGNOSE

1. Inspect HTTP status and response body
2. Check request fields and headers
3. Follow request ID into logs
4. Re-run the smallest failing test

LAB 03 · CONTRACT REVIEW

Prompt output must preserve the API boundary

TARGET FILES

```
app/main.py

app/routes.py

app/service.py

tests/test_routes.py
```

ACCEPTANCE

All baseline tests still pass and OpenAPI exposes each route exactly once.

ABILITY TRACE

A3, A8

LAB 04 · SCENARIO

Add API-Key Protection and CORS

DESIGN CHALLENGE

Implement a reusable API-key dependency and an explicit browser-origin allow list.

Build: security.py plus protected write routes

TOOLS

FastAPI Depends, Header, CORSMiddleware, OpenAI Python SDK

Detailed procedure: Learner Guide + lab README

Prompt submitted by ai_vibe.py

LEARNER PROMPT

```
You are working on Lab 04 of a Northstar Commerce API. Add a
reusable X-API-Key dependency to product, customer and order write
routes and configure CORS for only the origin supplied in mock-
data.json. Read secrets from environment variables, compare
safely, never place a live key in source or browser JavaScript,
and test missing, invalid and valid credentials.
```

PYTHON SDK ENVELOPE

- Prompts.pdf
- mock-data.json
- **responses.parse**
- **CodeBundle**
- generated/
- pytest + review

Full prompt + refinement
in the lab Prompts.pdf

LAB 04 · MECHANISM

How Add API-Key Protection and CORS works

1

Input contract

Implement a reusable API-key dependency and an explicit browser-origin allow list.

2

Boundary validation

Reject invalid or unauthorised input

3

Business decision

Apply one explicit rule

4

State/evidence

Unauthorized writes return 401; a valid key succeeds; only the configured origin receives CORS permission.

security.py plus protected write routes

```
REQUEST
  lab: 04
  objective: Implement a reusable API-key dependency and an
             explicit browser-origin allow list.

EXPECTED EVIDENCE
  artifact: security.py plus protected write routes
  codes: A7, A8
  status: verified
```

ASSESSMENT BRIDGE

Ability codes
A7, A8

Capture a screenshot, terminal output or test result that proves the acceptance criterion.

LAB 04 · VERIFY

Acceptance and diagnosis

PASS WHEN

Unauthorized writes return 401; a valid key succeeds; only the configured origin receives CORS permission.

DIAGNOSE

1. Inspect HTTP status and response body
2. Check request fields and headers
3. Follow request ID into logs
4. Re-run the smallest failing test

LAB 04 · CONTRACT REVIEW

Prompt output must preserve the API boundary

TARGET FILES

```
app/security.py

app/main.py

tests/test_security.py
```

ACCEPTANCE

Unauthorized writes return 401; a valid key succeeds; only the configured origin receives CORS permission.

ABILITY TRACE

A7, A8

TOPIC 03

Request Handling, Data Validation and JSON Responses

Path/query/body data · Pydantic models · response filtering · PATCH

03

FastAPI classifies inputs by declaration

A name in the path is a path parameter; a scalar is query data; a Pydantic model is a JSON body.

1. /products/{product_id}
2. ?status=open
3. ProductCreate body
4. Header for cross-cutting metadata

ARTIFACT Input-source map

FAILURE Ambiguous or duplicated fields create contradictory contracts.

VERIFY OpenAPI shows every input in the intended location.

Prove: FastAPI classifies inputs by declaration

INPUT

/products/{product_id}

MECHANISM

A name in the path is a path parameter; a scalar is query data; a Pydantic model is a JSON body.

FAILURE

Ambiguous or duplicated fields create contradictory contracts.

PROOF

OpenAPI shows every input in the intended location.

Evidence artifact: Input-source map

Pydantic validates at the boundary

Field types and constraints reject invalid data before business logic runs.

```
class ProductCreate(BaseModel):
    sku: str = Field(pattern=r"^[A-Z0-9-]+$")
    price: Decimal = Field(ge=0, decimal_places=2)
    stock_quantity: int = Field(ge=0)
```

- min_length
- pattern
- ge/le
- Literal/Enum

ARTIFACT ProductCreate schema

FAILURE Validation after SQL can persist invalid state.

VERIFY Boundary tests cover invalid SKU, negative price/stock and unsupported status.

Prove: Pydantic validates at the boundary

INPUT

min_length

MECHANISM

Field types and constraints reject invalid data before business logic runs.

FAILURE

Validation after SQL can persist invalid state.

PROOF

Boundary tests cover invalid SKU, negative price/stock and unsupported status.

Evidence artifact: ProductCreate schema

422 errors point to the exact invalid field

The error payload includes location, message and error type for each failed input.

1. loc: body.sku
2. msg: string pattern mismatch
3. type: string_pattern_mismatch
4. client can map error to UI

ARTIFACT Structured validation error JSON

FAILURE Replacing 422 with a generic 400 discards useful machine-readable detail.

VERIFY Frontend renders the failing field and keeps the user-entered values.

Prove: 422 errors point to the exact invalid field

INPUT

loc: body.sku

MECHANISM

The error payload includes location, message and error type for each failed input.

FAILURE

Replacing 422 with a generic 400 discards useful machine-readable detail.

PROOF

Frontend renders the failing field and keeps the user-entered values.

Evidence artifact: Structured validation error JSON

TOPIC 03 · FLOW ANCHOR

Response models are an output firewall

FastAPI validates and filters returned data against the declared output model.

1. handler returns object
2. response_model validates
3. private fields removed
4. JSON encoded

ARTIFACT ProductRow → ProductRead

FAILURE Returning database rows directly can leak internal columns.

VERIFY A test asserts secret/internal fields never appear in JSON.

Prove: Response models are an output firewall

INPUT

handler returns object

MECHANISM

FastAPI validates and filters returned data against the declared output model.

FAILURE

Returning database rows directly can leak internal columns.

PROOF

A test asserts secret/internal fields never appear in JSON.

Evidence artifact: ProductRow → ProductRead

JSON shape is part of compatibility

Clients depend on field names, types, optionality and nesting—not just values.

1. Stable: {items,total}
2. Breaking: rename title→name
3. Compatible: add optional field
4. Version breaking contracts

ARTIFACT Before/after response diff

FAILURE Silent schema drift breaks frontend code at runtime.

VERIFY Contract tests compare responses to the documented schema.

Prove: JSON shape is part of compatibility

INPUT

Stable: {items,total}

MECHANISM

Clients depend on field names, types, optionality and nesting—not just values.

FAILURE

Silent schema drift breaks frontend code at runtime.

PROOF

Contract tests compare responses to the documented schema.

Evidence artifact: Before/after response diff

PATCH requires an explicit missing-value rule

model_dump(exclude_unset=True) distinguishes omitted fields from fields explicitly set to null.

```
changes = patch.model_dump(exclude_unset=True)
updated = service.update(product_id, changes)
```

- omitted = keep current
- present = update
- null = allowed only where schema permits
- validate merged state

ARTIFACT Partial update dictionary

FAILURE Using model defaults overwrites fields the client did not send.

VERIFY Updating one field leaves all other stored fields unchanged.

Prove: PATCH requires an explicit missing-value rule

INPUT

omitted = keep current

MECHANISM

model_dump(exclude_unset=True) distinguishes omitted fields from fields explicitly set to null.

FAILURE

Using model defaults overwrites fields the client did not send.

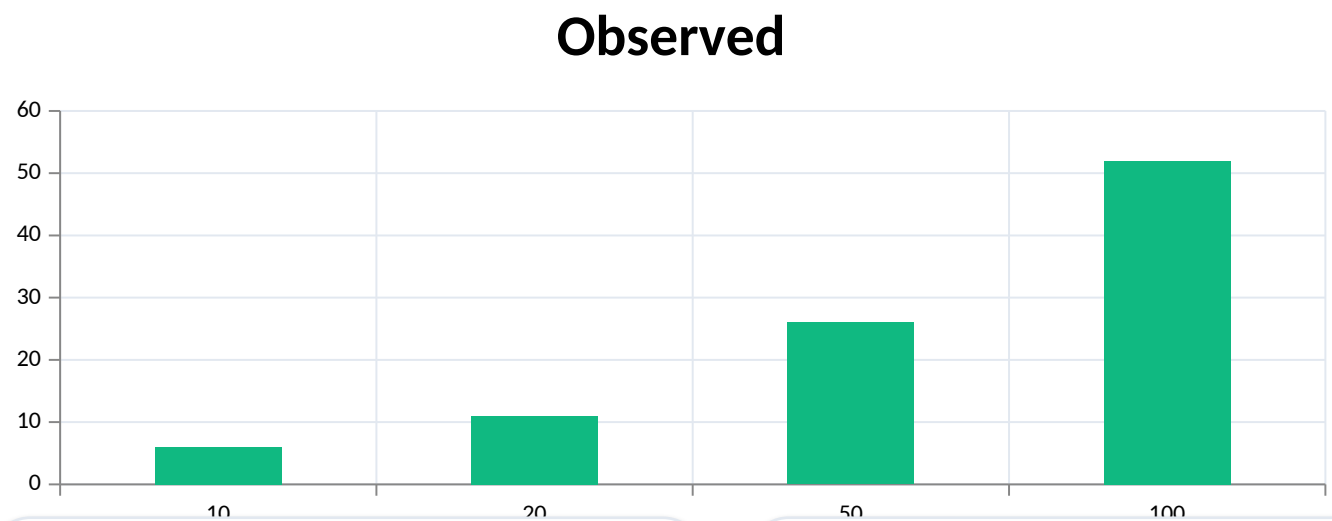
PROOF

Updating one field leaves all other stored fields unchanged.

Evidence artifact: Partial update dictionary

Pagination protects service and client

Limit and offset bound work; response metadata lets the UI navigate predictably.



01 limit default 20

02 cap limit at 100

03 offset >= 0

04 include total count

ARTIFACT Payload size by page limit

FAILURE Unbounded collection routes can exhaust memory and freeze browsers.

VERIFY A request above the cap is rejected or constrained as documented.

Prove: Pagination protects service and client

INPUT

limit default 20

MECHANISM

Limit and offset bound work; response metadata lets the UI navigate predictably.

FAILURE

Unbounded collection routes can exhaust memory and freeze browsers.

PROOF

A request above the cap is rejected or constrained as documented.

Evidence artifact: Payload size by page limit

Serialize dates and enums deliberately

Use typed datetime and enum fields so JSON encoding and OpenAPI remain consistent.

1. datetime → ISO 8601
2. Enum → stable string
3. decimal policy
4. timezone decision

ARTIFACT Python value ↔ JSON value map

FAILURE Naive local timestamps become ambiguous across clients.

VERIFY Round-trip tests preserve meaning across encode and decode.

Prove: Serialize dates and enums deliberately

INPUT

datetime → ISO 8601

MECHANISM

Use typed datetime and enum fields so JSON encoding and OpenAPI remain consistent.

FAILURE

Naive local timestamps become ambiguous across clients.

PROOF

Round-trip tests preserve meaning across encode and decode.

Evidence artifact: Python value ↔ JSON value map

LAB 05 · SCENARIO

Validate Product and Customer Data

DESIGN CHALLENGE

Define typed product and customer schemas with field-level and cross-field constraints.

Build: schemas.py with ProductCreate, ProductPatch and ProductRead

TOOLS

Pydantic v2, Swagger UI, OpenAI Python SDK

Detailed procedure: Learner Guide + lab README

Prompt submitted by ai_vibe.py

LEARNER PROMPT

```
You are working on Lab 05 of a Northstar Commerce API. Create
Pydantic v2 Product, Customer, Order and OrderItem input/output
models. Constrain SKU, email, Decimal price, stock quantity and
status; distinguish required, optional and omitted values. Add
schema examples and boundary tests that assert the exact 422 error
locations for invalid mock requests.
```

PYTHON SDK ENVELOPE

- Prompts.pdf
- mock-data.json
- **responses.parse**
- **CodeBundle**
- generated/
- pytest + review

Full prompt + refinement
in the lab Prompts.pdf

How Validate Product and Customer Data works

1

Input contract

Define typed product and customer schemas with field-level and cross-field constraints.

2

Boundary validation

Reject invalid or unauthorised input

3

Business decision

Apply one explicit rule

4

State/evidence

Invalid SKU, price, stock, email and status values are rejected before repository code; valid JSON becomes a typed model.

schemas.py with ProductCreate, ProductPatch and ProductRead

```
REQUEST
  lab: 05
  objective: Define typed product and customer schemas with
field-
             level and cross-field constraints.

EXPECTED EVIDENCE
  artifact: schemas.py with ProductCreate, ProductPatch and
ProductRead
  codes: A3, A5
  status: verified
```

ASSESSMENT BRIDGE

Ability codes
A3, A5

Capture a screenshot, terminal output or test result that proves the acceptance criterion.

Acceptance and diagnosis

PASS WHEN

Invalid SKU, price, stock, email and status values are rejected before repository code; valid JSON becomes a typed model.

DIAGNOSE

1. Inspect HTTP status and response body
2. Check request fields and headers
3. Follow request ID into logs
4. Re-run the smallest failing test

Prompt output must preserve the API boundary

TARGET FILES

`app/schemas.py`

`tests/test_validation.py`

ACCEPTANCE

Invalid SKU, price, stock, email and status values are rejected before repository code; valid JSON becomes a typed model.

ABILITY TRACE

A3, A5

LAB 06 · SCENARIO

Filter Responses and Implement PATCH

DESIGN CHALLENGE

Use response models and exclude_unset semantics for safe partial updates.

Build: PATCH /api/v1/products/{id} with stable ProductRead output

TOOLS

FastAPI response_model, Pydantic, OpenAI Python SDK

Detailed procedure: Learner Guide + lab README

Prompt submitted by ai_vibe.py

LEARNER PROMPT

```
You are working on Lab 06 of a Northstar Commerce API. Revise the
FastAPI product routes so response models exclude internal fields
and PATCH uses model_dump(exclude_unset=True). Preserve
unspecified fields, define the empty-patch rule and add tests for
one-field updates, response filtering, missing products and stable
JSON shapes.
```

PYTHON SDK ENVELOPE

- Prompts.pdf
- mock-data.json
- **responses.parse**
- **CodeBundle**
- generated/
- pytest + review

Full prompt + refinement
in the lab Prompts.pdf

How Filter Responses and Implement PATCH works

1

Input contract

Use response models and exclude_unset semantics for safe partial updates.

2

Boundary validation

Reject invalid or unauthorised input

3

Business decision

Apply one explicit rule

4

State/evidence

PATCH changes only supplied fields and no internal database field appears in the JSON response.

PATCH /api/v1/products/{id} with stable ProductRead output

```
REQUEST
  lab: 06
  objective: Use response models and exclude_unset semantics
for
             safe partial updates.

EXPECTED EVIDENCE
  artifact: PATCH /api/v1/products/{id} with stable ProductRead
output
  codes: A4, A8
  status: verified
```

ASSESSMENT BRIDGE

Ability codes
A4, A8

Capture a screenshot, terminal output or test result that proves the acceptance criterion.

Acceptance and diagnosis

PASS WHEN

PATCH changes only supplied fields and no internal database field appears in the JSON response.

DIAGNOSE

1. Inspect HTTP status and response body
2. Check request fields and headers
3. Follow request ID into logs
4. Re-run the smallest failing test

Prompt output must preserve the API boundary

TARGET FILES

```
app/main.py

app/schemas.py

tests/test_patch.py
```

ACCEPTANCE

PATCH changes only supplied fields and no internal database field appears in the JSON response.

ABILITY TRACE

A4, A8

TOPIC 04

API Error Handling, Testing and Debugging

HTTPException · logs · TestClient · isolation · failure diagnosis

04

Errors are designed outcomes

Expected client failures use HTTPException; unexpected failures are logged and mapped without leaking internals.

1. 404 missing
2. 409 state conflict
3. 422 validation
4. 500 unexpected

ARTIFACT Failure taxonomy

FAILURE Catching every Exception and returning 200 hides defects.

VERIFY Each documented failure has a deterministic status and response body.

Prove: Errors are designed outcomes

INPUT

404 missing

MECHANISM

Expected client failures use HTTPException; unexpected failures are logged and mapped without leaking internals.

FAILURE

Catching every Exception and returning 200 hides defects.

PROOF

Each documented failure has a deterministic status and response body.

Evidence artifact: Failure taxonomy

Raise HTTPException at the decision point

Stop processing when a resource or rule fails and return a meaningful detail.

```
product = repo.get(product_id)
if product is None:
    raise HTTPException(404, "Product not found")
```

- lookup
- decide
- raise
- framework serializes

ARTIFACT 404 response path

FAILURE Returning None may later cause a confusing serialization or attribute error.

VERIFY Missing ID returns 404 and performs no write.

Prove: Raise HTTPException at the decision point

INPUT

lookup

MECHANISM

Stop processing when a resource or rule fails and return a meaningful detail.

FAILURE

Returning None may later cause a confusing serialization or attribute error.

PROOF

Missing ID returns 404 and performs no write.

Evidence artifact: 404 response path

Custom handlers standardise error shape

An exception handler converts different internal failures into a consistent public envelope.

ARTIFACT {error: {code,message,request_id}}

FAILURE Leaking stack traces exposes file paths and implementation details.

VERIFY Clients parse one stable error envelope while logs retain diagnostic context.

Prove: Custom handlers standardise error shape

INPUT

exception

MECHANISM

An exception handler converts different internal failures into a consistent public envelope.

FAILURE

Leaking stack traces exposes file paths and implementation details.

PROOF

Clients parse one stable error envelope while logs retain diagnostic context.

Evidence artifact: {error:{code,message,request_id}}

Tests follow Arrange–Act–Assert

Arrange state, perform one HTTP interaction, then assert transport and business evidence.

1. Arrange test DB
2. Act with TestClient
3. Assert status
4. Assert JSON
5. Assert persisted state

ARTIFACT pytest test case

FAILURE A test that asserts only status may miss corrupt data.

VERIFY Each write test checks response and database state.

Prove: Tests follow Arrange–Act–Assert

INPUT

Arrange test DB

MECHANISM

Arrange state, perform one HTTP interaction, then assert transport and business evidence.

FAILURE

A test that asserts only status may miss corrupt data.

PROOF

Each write test checks response and database state.

Evidence artifact: pytest test case

TestClient exercises the ASGI application

FastAPI's TestClient sends requests without a live network server.

```
client = TestClient(app)
r = client.post("/api/v1/products", json={"sku":"NS-
MUG-02","name":"Field Notes Mug","price":"26.00"})
assert r.status_code == 201
```

- same routes
- same validation
- fast deterministic loop
- pytest-friendly

ARTIFACT client.post('/api/v1/products', json=...)

FAILURE Manual Swagger checks are useful exploration but not regression protection.

VERIFY The suite runs unattended and reports repeatable pass/fail evidence.

Prove: TestClient exercises the ASGI application

INPUT

same routes

MECHANISM

FastAPI's TestClient sends requests without a live network server.

FAILURE

Manual Swagger checks are useful exploration but not regression protection.

PROOF

The suite runs unattended and reports repeatable pass/fail evidence.

Evidence artifact: client.post('/api/v1/products', json=...)

Isolate test data

Tests need a temporary SQLite database and dependency override so runs cannot alter demo data.

ARTIFACT production DB dependency ↔ test override

FAILURE Tests against the real database are destructive and order-dependent.

VERIFY A test run leaves the demo database checksum unchanged.

Prove: Isolate test data

INPUT

temporary path

MECHANISM

Tests need a temporary SQLite database and dependency override so runs cannot alter demo data.

FAILURE

Tests against the real database are destructive and order-dependent.

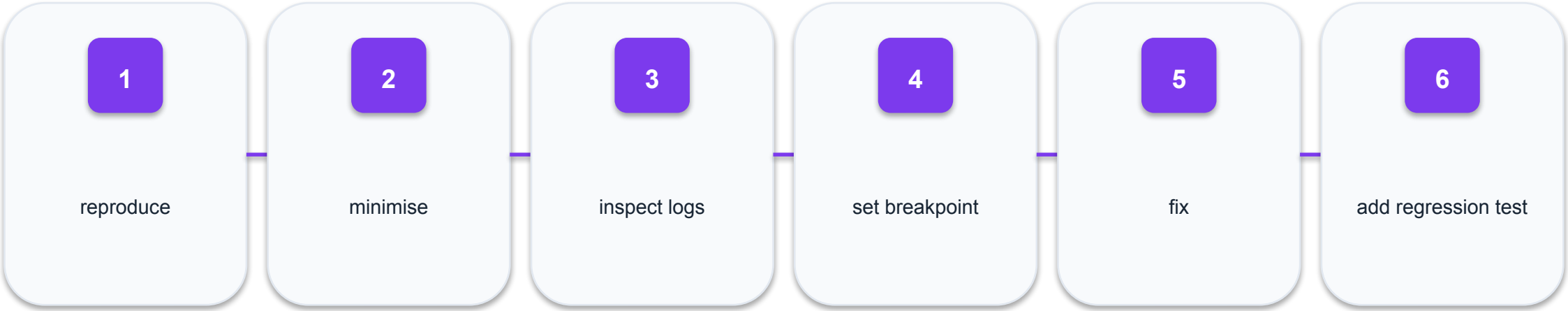
PROOF

A test run leaves the demo database checksum unchanged.

Evidence artifact: production DB dependency ↔ test override

Debug from evidence, not guesses

Follow request ID, status, log event, stack location, input and database state in order.



ARTIFACT Failure-to-fix trace

FAILURE Changing multiple things at once destroys causal evidence.

VERIFY The regression test fails before the fix and passes after it.

Prove: Debug from evidence, not guesses

INPUT

reproduce

MECHANISM

Follow request ID, status, log event, stack location, input and database state in order.

FAILURE

Changing multiple things at once destroys causal evidence.

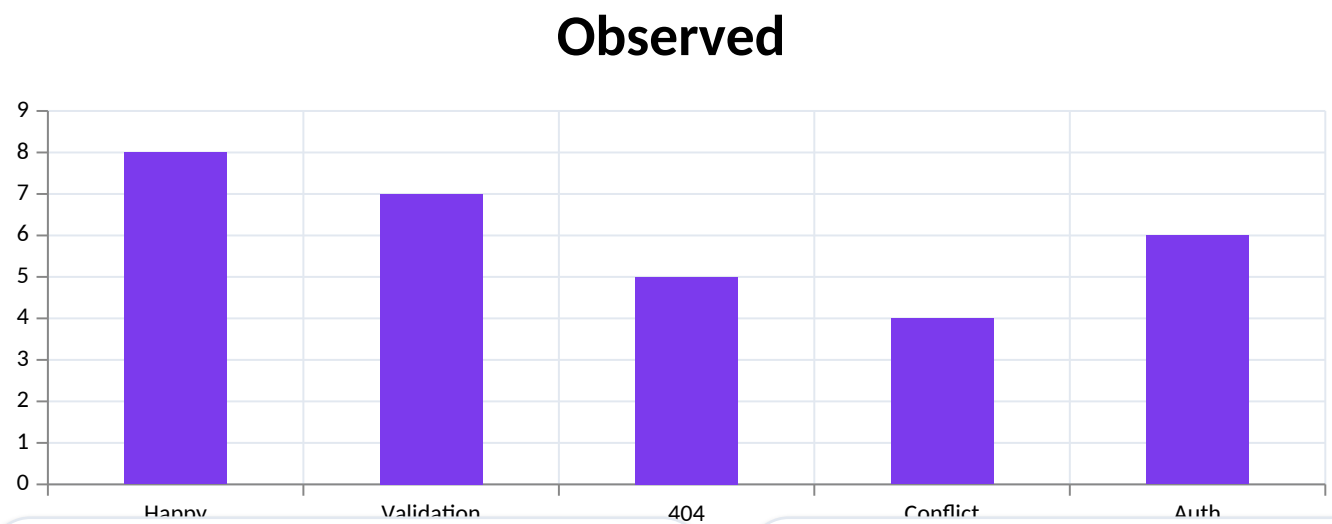
PROOF

The regression test fails before the fix and passes after it.

Evidence artifact: Failure-to-fix trace

A test portfolio covers multiple risks

Balance happy paths, validation, not-found, conflict and authorization tests.



01 **functional**

02 boundary

03 negative

04 security

ARTIFACT Test distribution by risk

FAILURE A suite dominated by happy paths creates false confidence.

VERIFY Every route has at least one success and one failure assertion.

Prove: A test portfolio covers multiple risks

INPUT

functional

MECHANISM

Balance happy paths, validation, not-found, conflict and authorization tests.

FAILURE

A suite dominated by happy paths creates false confidence.

PROOF

Every route has at least one success and one failure assertion.

Evidence artifact: Test distribution by risk

LAB 07 · SCENARIO

Standardise Errors and Request Logs

DESIGN CHALLENGE

Create predictable error responses and trace a request from status to log event.

Build: error envelope and request-ID middleware

TOOLS

HTTPException, logging, middleware, OpenAI Python SDK

Detailed procedure: Learner Guide + lab README

Prompt submitted by ai_vibe.py

LEARNER PROMPT

```
You are working on Lab 07 of a Northstar Commerce API. Generate
request-ID middleware, structured request logging and one public
error envelope for 404 and 409 outcomes. Logs may include method,
path, status, request id and elapsed time but no API key or body
secret. Add tests that correlate the response request id with
captured log evidence.
```

PYTHON SDK ENVELOPE

- Prompts.pdf
- mock-data.json
- **responses.parse**
- **CodeBundle**
- generated/
- pytest + review

Full prompt + refinement
in the lab Prompts.pdf

How Standardise Errors and Request Logs works

1

Input contract

Create predictable error responses and trace a request from status to log event.

2

Boundary validation

Reject invalid or unauthorised input

3

Business decision

Apply one explicit rule

4

State/evidence

404 and 409 use the same envelope; each response request ID locates one log trace; secrets are absent.

error envelope and request-ID middleware

```
REQUEST
  lab: 07
  objective: Create predictable error responses and trace a
request
             from status to log event.

EXPECTED EVIDENCE
  artifact: error envelope and request-ID middleware
  codes: A5, A7
  status: verified
```

ASSESSMENT BRIDGE

Ability codes
A5, A7

Capture a screenshot, terminal output or test result that proves the acceptance criterion.

LAB 07 · VERIFY

Acceptance and diagnosis

PASS WHEN

404 and 409 use the same envelope; each response request ID locates one log trace; secrets are absent.

DIAGNOSE

1. Inspect HTTP status and response body
2. Check request fields and headers
3. Follow request ID into logs
4. Re-run the smallest failing test

Prompt output must preserve the API boundary

TARGET FILES

`app/main.py`

`app/errors.py`

`tests/test_errors.py`

ACCEPTANCE

404 and 409 use the same envelope; each response request ID locates one log trace; secrets are absent.

ABILITY TRACE

A5, A7

LAB 08 · SCENARIO

Test and Debug the API

DESIGN CHALLENGE

Build a pytest regression suite, reproduce a seeded defect and verify the fix.

Build: tests/test_commerce.py with success and failure coverage

TOOLS

pytest, FastAPI TestClient, VS Code debugger, OpenAI Python SDK

Detailed procedure: Learner Guide + lab README

Prompt submitted by ai_vibe.py

LEARNER PROMPT

```
You are working on Lab 08 of a Northstar Commerce API. Using the
seeded failing cases in mock-data.json, produce a focused pytest
regression suite and a debug plan. Reproduce the failure with the
smallest request, assert response and persisted state, identify
the likely boundary, and propose the minimum patch without
weakening existing acceptance tests.
```

PYTHON SDK ENVELOPE

- Prompts.pdf
- mock-data.json
- **responses.parse**
- **CodeBundle**
- generated/
- pytest + review

Full prompt + refinement
in the lab Prompts.pdf

How Test and Debug the API works

1

Input contract

Build a pytest regression suite, reproduce a seeded defect and verify the fix.

2

Boundary validation

Reject invalid or unauthorised input

3

Business decision

Apply one explicit rule

4

State/evidence

The seeded test fails before the fix; the full suite passes after it; the regression assertion remains.

tests/test_commerce.py with success and failure coverage

```
REQUEST
  lab: 08
  objective: Build a pytest regression suite, reproduce a
seeded
             defect and verify the fix.

EXPECTED EVIDENCE
  artifact: tests/test_commerce.py with success and failure
coverage
  codes: A5, A6, A7
  status: verified
```

ASSESSMENT BRIDGE

Ability codes
A5, A6, A7

Capture a screenshot, terminal output or test result that proves the acceptance criterion.

LAB 08 · VERIFY

Acceptance and diagnosis

PASS WHEN

The seeded test fails before the fix; the full suite passes after it; the regression assertion remains.

DIAGNOSE

1. Inspect HTTP status and response body
2. Check request fields and headers
3. Follow request ID into logs
4. Re-run the smallest failing test

Prompt output must preserve the API boundary

TARGET FILES

`tests/test_regression.py`

`debug-plan.md`

ACCEPTANCE

The seeded test fails before the fix; the full suite passes after it; the regression assertion remains.

ABILITY TRACE

A5, A6, A7

TOPIC 05

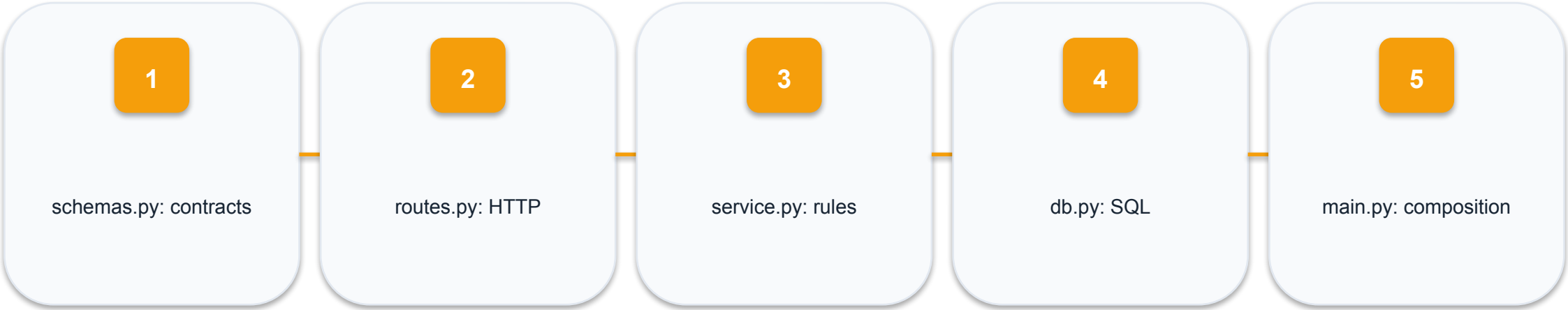
API Architecture, Data Models and Database Integration

Layered design · SQLite · parameterized SQL · transactions · CRUD

05

Layering localises change

Schemas, routes, services and repositories change for different reasons.



ARTIFACT Five-file application boundary

FAILURE A monolithic app.py entangles tests and makes AI edits broad and risky.

VERIFY A schema change touches the smallest justified set of files.

Prove: Layering localises change

INPUT

schemas.py: contracts

MECHANISM

Schemas, routes, services and repositories change for different reasons.

FAILURE

A monolithic app.py entangles tests and makes AI edits broad and risky.

PROOF

A schema change touches the smallest justified set of files.

Evidence artifact: Five-file application boundary

SQLite is a serverless relational engine

The database is a local file accessed by the Python process; it still enforces tables, keys and transactions.

1. single file
2. ACID transactions
3. SQL queries
4. excellent demo/local fit

ARTIFACT	northstar.db with products table
FAILURE	Treating SQLite as an unstructured file loses relational guarantees.
VERIFY	Schema inspection shows primary key, defaults and constraints.

Prove: SQLite is a serverless relational engine

INPUT

single file

MECHANISM

The database is a local file accessed by the Python process; it still enforces tables, keys and transactions.

FAILURE

Treating SQLite as an unstructured file loses relational guarantees.

PROOF

Schema inspection shows primary key, defaults and constraints.

Evidence artifact: northstar.db with products table

Design the table around invariants

Use constraints and defaults to protect state even when a code path is wrong.

1. Product and Customer keys
2. OrderItem foreign keys
3. money/stock CHECK
4. unique SKU/email

ARTIFACT Commerce relational schema

FAILURE Application-only validation can be bypassed by another writer.

VERIFY Invalid direct SQL writes fail at the database boundary.

Prove: Design the table around invariants

INPUT

Product and Customer keys

MECHANISM

Use constraints and defaults to protect state even when a code path is wrong.

FAILURE

Application-only validation can be bypassed by another writer.

PROOF

Invalid direct SQL writes fail at the database boundary.

Evidence artifact: Commerce relational schema

Parameterized SQL separates code from data

Use placeholders and parameter tuples; never concatenate user input into SQL.

```
row = conn.execute(
    "SELECT * FROM products WHERE id = ?",
(product_id,)
).fetchone()
```

- query template fixed
- values bound separately
- quotes handled safely
- injection payload stays data

ARTIFACT SELECT ... WHERE id = ?

FAILURE f-string SQL allows input to change query structure.

VERIFY An injection-shaped product name is stored or rejected as data, never executed.

Prove: Parameterized SQL separates code from data

INPUT

query template fixed

MECHANISM

Use placeholders and parameter tuples; never concatenate user input into SQL.

FAILURE

f-string SQL allows input to change query structure.

PROOF

An injection-shaped product name is stored or rejected as data, never executed.

Evidence artifact: SELECT ... WHERE id = ?

Transactions define atomic state changes

A write either commits as a complete unit or rolls back.

ARTIFACT Request-scoped connection context

FAILURE Partial writes leave contradictory state when an exception interrupts work.

VERIFY A forced failure leaves the database in its pre-request state.

Prove: Transactions define atomic state changes

INPUT

BEGIN

MECHANISM

A write either commits as a complete unit or rolls back.

FAILURE

Partial writes leave contradictory state when an exception interrupts work.

PROOF

A forced failure leaves the database in its pre-request state.

Evidence artifact: Request-scoped connection context

CRUD maps contracts to SQL

Each endpoint has a predictable statement, success code and missing-resource path.

1 POST→INSERT→201

2 GET→SELECT→200/404

3 PATCH→UPDATE→200/404

4 DELETE→DELETE→204/404

ARTIFACT Endpoint-to-SQL matrix

FAILURE A DELETE that always returns 204 cannot distinguish an absent record.

VERIFY CRUD integration tests cover both existing and missing IDs.

Prove: CRUD maps contracts to SQL

INPUT

POST→INSERT→201

MECHANISM

Each endpoint has a predictable statement, success code and missing-resource path.

FAILURE

A DELETE that always returns 204 cannot distinguish an absent record.

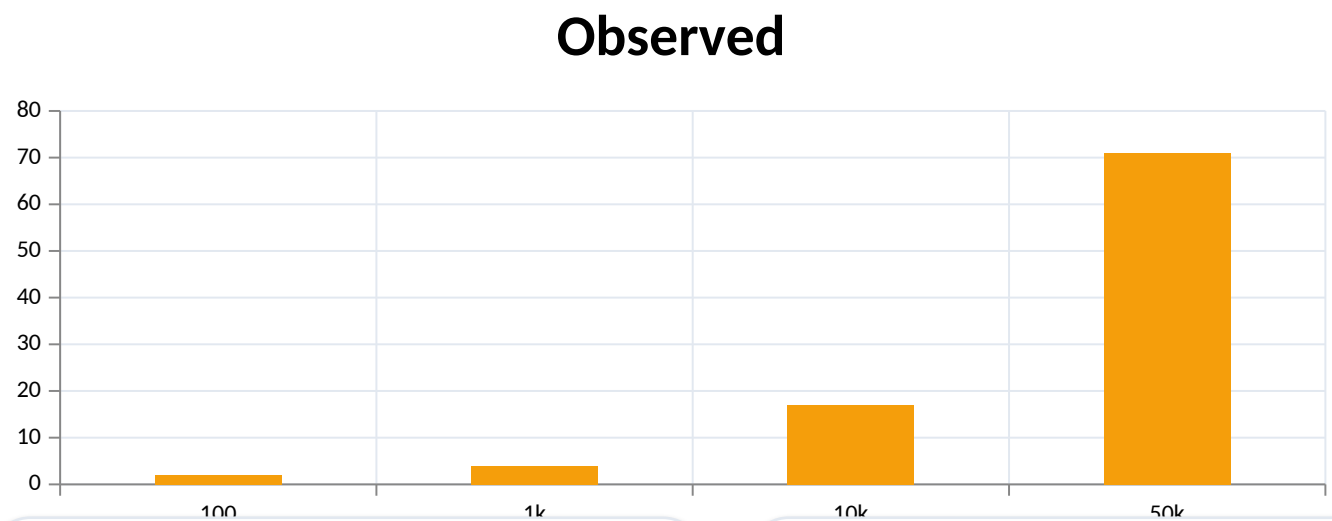
PROOF

CRUD integration tests cover both existing and missing IDs.

Evidence artifact: Endpoint-to-SQL matrix

Indexes trade write work for read speed

An index on frequently filtered columns reduces scanning as rows grow.



01 index status

02 measure query plan

03 avoid indexing every field

04 small demos may not need many

ARTIFACT Measured query time by row count

FAILURE Premature indexes add write overhead and complexity without evidence.

VERIFY EXPLAIN QUERY PLAN shows the expected index for the target query.

Prove: Indexes trade write work for read speed

INPUT

index status

MECHANISM

An index on frequently filtered columns reduces scanning as rows grow.

FAILURE

Premature indexes add write overhead and complexity without evidence.

PROOF

EXPLAIN QUERY PLAN shows the expected index for the target query.

Evidence artifact: Measured query time by row count

Connection lifetime is an ownership decision

Open a short-lived request-scoped connection, configure row access and foreign keys, then close it reliably.

1. Context manager
2. row_factory
3. PRAGMA foreign_keys=ON
4. commit/rollback

ARTIFACT Connection lifecycle

FAILURE A leaked connection or long transaction holds locks and creates intermittent failures.

VERIFY Load tests show no growing open-resource count and no locked-database errors.

Prove: Connection lifetime is an ownership decision

INPUT

Context manager

MECHANISM

Open a short-lived request-scoped connection, configure row access and foreign keys, then close it reliably.

FAILURE

A leaked connection or long transaction holds locks and creates intermittent failures.

PROOF

Load tests show no growing open-resource count and no locked-database errors.

Evidence artifact: Connection lifecycle

LAB 09 · SCENARIO

Design the Commerce CRM Database

DESIGN CHALLENGE

Create constrained products, customers, orders and order_items tables with relational integrity.

Build: db.py and schema.sql

TOOLS

Python sqlite3, SQLite CLI, OpenAI Python SDK

Detailed procedure: Learner Guide + lab README

Prompt submitted by ai_vibe.py

LEARNER PROMPT

```
You are working on Lab 09 of a Northstar Commerce API. Design
SQLite products, customers, orders and order_items tables with
primary/foreign keys, UNIQUE email/SKU, money and stock checks,
order-state checks and indexes. Generate an idempotent init_db
function using sqlite3.Row and PRAGMA foreign_keys=ON, plus tests
that initialise twice and reject invalid relationships.
```

PYTHON SDK ENVELOPE

- Prompts.pdf
- mock-data.json
- **responses.parse**
- **CodeBundle**
- generated/
- pytest + review

Full prompt + refinement
in the lab Prompts.pdf

How Design the Commerce CRM Database works

1

Input contract

Create constrained products, customers, orders and order_items tables with relational integrity.

2

Boundary validation

Reject invalid or unauthorised input

3

Business decision

Apply one explicit rule

4

State/evidence

Initialisation is idempotent and SQLite enforces unique SKU/email, valid money/stock, order status and foreign keys.

db.py and schema.sql

```
REQUEST
  lab: 09
  objective: Create constrained products, customers, orders and
             order_items tables with relational
             integrity.

EXPECTED EVIDENCE
  artifact: db.py and schema.sql
  codes: A1, A3
  status: verified
```

ASSESSMENT BRIDGE

Ability codes
A1, A3

Capture a screenshot, terminal output or test result that proves the acceptance criterion.

Acceptance and diagnosis

PASS WHEN

Initialisation is idempotent and SQLite enforces unique SKU/email, valid money/stock, order status and foreign keys.

DIAGNOSE

1. Inspect HTTP status and response body
2. Check request fields and headers
3. Follow request ID into logs
4. Re-run the smallest failing test

Prompt output must preserve the API boundary

TARGET FILES

```
app/schema.sql

app/db.py

tests/test_database.py
```

ACCEPTANCE

Initialisation is idempotent and SQLite enforces unique SKU/email, valid money/stock, order status and foreign keys.

ABILITY TRACE

A1, A3

LAB 10 · SCENARIO

Implement SQLite CRUD and Orders Safely

DESIGN CHALLENGE

Connect product, customer and order operations to parameterized SQL and atomic transactions.

Build: repository.py with catalog, CRM and order transaction functions

TOOLS

sqlite3, FastAPI, pytest, OpenAI Python SDK

Detailed procedure: Learner Guide + lab README

Prompt submitted by ai_vibe.py

LEARNER PROMPT

```
You are working on Lab 10 of a Northstar Commerce API. Implement
parameterized SQLite repository functions for product/customer
CRUD and atomic order creation. Validate stock inside the same
transaction, insert order items, decrement inventory or roll back
fully, use bounded pagination and an allow-list for PATCH columns.
Add CRUD, insufficient-stock and injection-shaped-input tests.
```

PYTHON SDK ENVELOPE

- Prompts.pdf
- mock-data.json
- **responses.parse**
- **CodeBundle**
- generated/
- pytest + review

Full prompt + refinement
in the lab Prompts.pdf

How Implement SQLite CRUD and Orders Safely works

1

Input contract

Connect product, customer and order operations to parameterized SQL and atomic transactions.

2

Boundary validation

Reject invalid or unauthorised input

3

Business decision

Apply one explicit rule

4

State/evidence

Catalog/CRM CRUD and order tests pass, insufficient stock rolls back fully, missing IDs map to 404 and SQL values are parameter-bound.

repository.py with catalog, CRM and order transaction functions

```
REQUEST
  lab: 10
  objective: Connect product, customer and order operations to
             parameterized SQL and atomic
             transactions.

EXPECTED EVIDENCE
  artifact: repository.py with catalog, CRM and order
transaction functions
  codes: A3, A4, A5
  status: verified
```

ASSESSMENT BRIDGE

Ability codes
A3, A4, A5

Capture a screenshot, terminal output or test result that proves the acceptance criterion.

LAB 10 · VERIFY

Acceptance and diagnosis

PASS WHEN

Catalog/CRM CRUD and order tests pass, insufficient stock rolls back fully, missing IDs map to 404 and SQL values are parameter-bound.

DIAGNOSE

1. Inspect HTTP status and response body
2. Check request fields and headers
3. Follow request ID into logs
4. Re-run the smallest failing test

Prompt output must preserve the API boundary

TARGET FILES

`app/repository.py`

`tests/test_repository.py`

ACCEPTANCE

Catalog/CRM CRUD and order tests pass, insufficient stock rolls back fully, missing IDs map to 404 and SQL values are parameter-bound.

ABILITY TRACE

A3, A4, A5

TOPIC 06

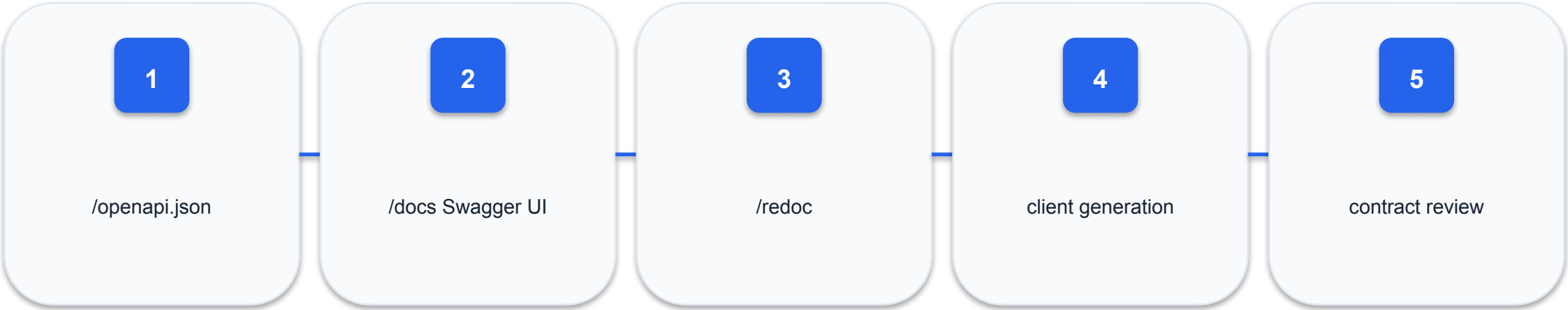
REST API Documentation, Integration and Deployment

OpenAPI · browser fetch · static frontend · versioning · packaging

06

OpenAPI is an executable interface inventory

FastAPI derives paths, parameters, schemas, responses and security metadata from code.



ARTIFACT Code annotations → OpenAPI → interactive docs

FAILURE Undocumented exceptions and security rules make docs misleading.

VERIFY A reviewer can execute every public operation from /docs.

Prove: OpenAPI is an executable interface inventory

INPUT

/openapi.json

MECHANISM

FastAPI derives paths, parameters, schemas, responses and security metadata from code.

FAILURE

Undocumented exceptions and security rules make docs misleading.

PROOF

A reviewer can execute every public operation from /docs.

Evidence artifact: Code annotations → OpenAPI → interactive docs

Operation metadata improves usability

Tags, summaries, descriptions, response codes and examples turn generated docs into a usable contract.

```
@router.post("/", response_model=ProductRead,
status_code=201,
    summary="Create a product", responses={409:
{"description":"Duplicate"}})
```

- tags
- summary
- status_code
- responses

ARTIFACT Annotated create-product operation

FAILURE Generic function names and missing response descriptions force guesswork.

VERIFY The docs distinguish create, validation, conflict and authorization outcomes.

Prove: Operation metadata improves usability

INPUT

tags

MECHANISM

Tags, summaries, descriptions, response codes and examples turn generated docs into a usable contract.

FAILURE

Generic function names and missing response descriptions force guesswork.

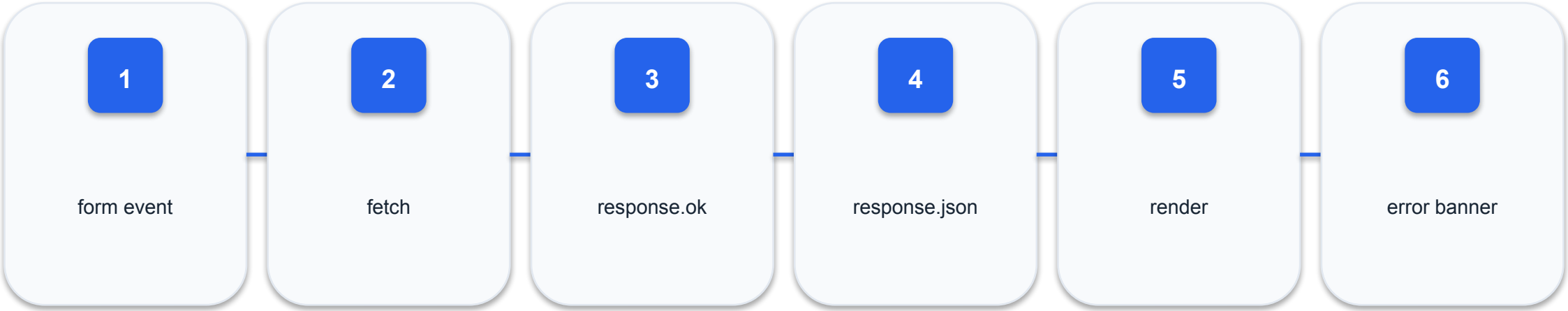
PROOF

The docs distinguish create, validation, conflict and authorization outcomes.

Evidence artifact: Annotated create-product operation

The browser client uses fetch as an API adapter

JavaScript serializes JSON, sends HTTP, checks status, parses JSON and renders DOM state.



ARTIFACT HTML form → JS adapter → FastAPI → SQLite

FAILURE Calling response.json() before checking 204 or non-JSON errors causes secondary failures.

VERIFY Network panel and visible UI agree on status and payload.

Prove: The browser client uses fetch as an API adapter

INPUT

form event

MECHANISM

JavaScript serializes JSON, sends HTTP, checks status, parses JSON and renders DOM state.

FAILURE

Calling response.json() before checking 204 or non-JSON errors causes secondary failures.

PROOF

Network panel and visible UI agree on status and payload.

Evidence artifact: HTML form → JS adapter → FastAPI → SQLite

Keep the frontend state derived from API responses

Render the server-returned product instead of assuming the requested change succeeded.

1. disable during request
2. use response body
3. refresh list
4. show error detail

ARTIFACT UI state transition model

FAILURE Optimistic updates without rollback leave the page inconsistent.

VERIFY Simulated 409/500 responses do not corrupt the visible product list.

Prove: Keep the frontend state derived from API responses

INPUT

disable during request

MECHANISM

Render the server-returned product instead of assuming the requested change succeeded.

FAILURE

Optimistic updates without rollback leave the page inconsistent.

PROOF

Simulated 409/500 responses do not corrupt the visible product list.

Evidence artifact: UI state transition model

Same-origin hosting removes demo CORS friction

Mount static files in FastAPI for a single-origin demo; configure explicit CORS only when origins differ.

1. Same origin: / + /api
2. Separate dev origin: 5500 + 8000
3. Explicit allow list
4. never put secret in browser

ARTIFACT Deployment topology comparison

FAILURE A frontend API key embedded in JavaScript is public by definition.

VERIFY Browser loads assets and API calls without blocked preflight or exposed secrets.

Prove: Same-origin hosting removes demo CORS friction

INPUT

Same origin: / + /api

MECHANISM

Mount static files in FastAPI for a single-origin demo; configure explicit CORS only when origins differ.

FAILURE

A frontend API key embedded in JavaScript is public by definition.

PROOF

Browser loads assets and API calls without blocked preflight or exposed secrets.

Evidence artifact: Deployment topology comparison

Health endpoints separate liveness from readiness

Liveness says the process responds; readiness proves required dependencies can serve traffic.

1. /health/live → process
2. /health/ready → database SELECT 1
3. fast and dependency-light
4. no sensitive details

ARTIFACT	Health-check contract
FAILURE	A single ‘OK’ endpoint can report healthy while the database is unavailable.
VERIFY	Stopping database access changes readiness but not liveness.

Prove: Health endpoints separate liveness from readiness

INPUT

/health/live → process

MECHANISM

Liveness says the process responds; readiness proves required dependencies can serve traffic.

FAILURE

A single ‘OK’ endpoint can report healthy while the database is unavailable.

PROOF

Stopping database access changes readiness but not liveness.

Evidence artifact: Health-check contract

Package a reproducible release

Pin dependencies, ignore secrets/data, initialise schema deterministically and document one start command.

1. requirements.txt
2. .env.example
3. .gitignore
4. schema init
5. tests
6. README

ARTIFACT Release bundle checklist

FAILURE Shipping a local .env or northstar.db leaks secrets and test data.

VERIFY A clean clone installs, tests and starts using only documented steps.

Prove: Package a reproducible release

INPUT

requirements.txt

MECHANISM

Pin dependencies, ignore secrets/data, initialise schema deterministically and document one start command.

FAILURE

Shipping a local .env or northstar.db leaks secrets and test data.

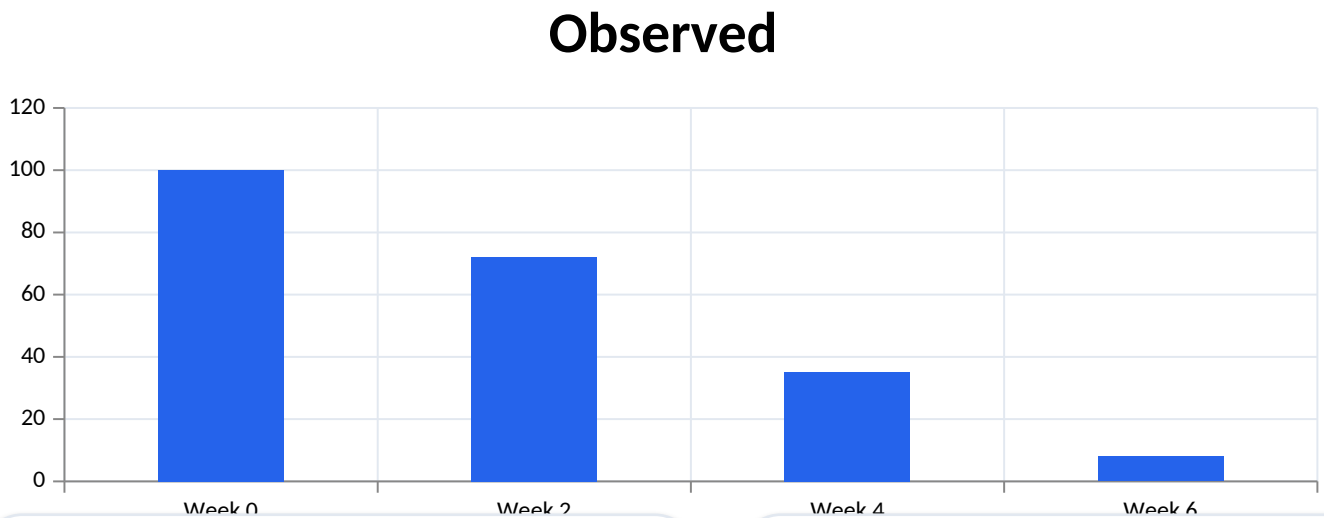
PROOF

A clean clone installs, tests and starts using only documented steps.

Evidence artifact: Release bundle checklist

API versioning manages breaking change

Use /api/v1 for a stable contract and introduce v2 only when compatibility cannot be preserved.



01 additive change first

02 deprecation window

03 usage telemetry

04 migration guide

ARTIFACT Client adoption across a deprecation window

FAILURE Changing v1 response fields in place silently breaks consumers.

VERIFY Both versions pass their contract suites until v1 retirement.

Prove: API versioning manages breaking change

INPUT

additive change first

MECHANISM

Use /api/v1 for a stable contract and introduce v2 only when compatibility cannot be preserved.

FAILURE

Changing v1 response fields in place silently breaks consumers.

PROOF

Both versions pass their contract suites until v1 retirement.

Evidence artifact: Client adoption across a deprecation window

LAB 11 · SCENARIO

Connect the Commerce Web App

DESIGN CHALLENGE

Use fetch to power a responsive dashboard for products, customers and orders through FastAPI.

Build: static/index.html, styles.css and app.js

TOOLS

HTML, CSS, JavaScript fetch, browser devtools, imagegen UI concept, OpenAI Python SDK

Detailed procedure: Learner Guide + lab README

Prompt submitted by ai_vibe.py

LEARNER PROMPT

```
You are working on Lab 11 of a Northstar Commerce API. Generate an
accessible responsive HTML/CSS/JavaScript Northstar Commerce
dashboard that uses fetch with the supplied FastAPI contract.
Implement product/customer creation, order placement, KPI refresh
and inventory rendering without page reload; keep live secrets out
of committed code; render structured errors and
empty/loading/failure states.
```

PYTHON SDK ENVELOPE

- Prompts.pdf
- mock-data.json
- **responses.parse**
- **CodeBundle**
- generated/
- pytest + review

Full prompt + refinement
in the lab Prompts.pdf

LAB 11 · MECHANISM

How Connect the Commerce Web App works

1

Input contract

Use fetch to power a responsive dashboard for products, customers and orders through FastAPI.

2

Boundary validation

Reject invalid or unauthorised input

3

Business decision

Apply one explicit rule

4

State/evidence

The browser reads/writes only through FastAPI, SQLite state changes correctly and the UI remains usable at 375px and desktop widths.

LAB 11 · EVIDENCE

static/index.html, styles.css and app.js

```
REQUEST
  lab: 11
  objective: Use fetch to power a responsive dashboard for
             products, customers and orders through
             FastAPI.

EXPECTED EVIDENCE
  artifact: static/index.html, styles.css and app.js
  codes: A3, A4, A6
  status: verified
```

ASSESSMENT BRIDGE

Ability codes
A3, A4, A6

Capture a screenshot, terminal output or test result that proves the acceptance criterion.

LAB 11 · VERIFY

Acceptance and diagnosis

PASS WHEN

The browser reads/writes only through FastAPI, SQLite state changes correctly and the UI remains usable at 375px and desktop widths.

DIAGNOSE

1. Inspect HTTP status and response body
2. Check request fields and headers
3. Follow request ID into logs
4. Re-run the smallest failing test

LAB 11 · CONTRACT REVIEW

Prompt output must preserve the API boundary

TARGET FILES

```
static/index.html

static/styles.css

static/app.js
```

ACCEPTANCE

The browser reads/writes only through FastAPI, SQLite state changes correctly and the UI remains usable at 375px and desktop widths.

ABILITY TRACE

A3, A4, A6

LAB 12 · SCENARIO

Document and Bundle the Capstone

DESIGN CHALLENGE

Produce a clean, testable release bundle with API docs, frontend assets and operational instructions.

Build: release-ready Northstar Commerce API

TOOLS

OpenAPI, pytest, Git, Uvicorn, OpenAI Python SDK

Detailed procedure: Learner Guide + lab README

Prompt submitted by ai_vibe.py

LEARNER PROMPT

```
You are working on Lab 12 of a Northstar Commerce API. Review the
complete FastAPI/SQLite/browser project and generate release
documentation only. Include clean setup, test, run and
verification commands; OpenAPI and environment notes; a bundle
inventory; and a checklist excluding .env, databases, caches and
virtual environments. Do not invent passing evidence.
```

PYTHON SDK ENVELOPE

- Prompts.pdf
- mock-data.json
- **responses.parse**
- **CodeBundle**
- generated/
- pytest + review

Full prompt + refinement
in the lab Prompts.pdf

How Document and Bundle the Capstone works

1

Input contract

Produce a clean, testable release bundle with API docs, frontend assets and operational instructions.

2

Boundary validation

Reject invalid or unauthorised input

3

Business decision

Apply one explicit rule

4

State/evidence

A clean copy installs, tests, starts, serves the storefront/CRM dashboard, completes catalog/customer/order flows and contains no secret or database file.

LAB 12 · EVIDENCE

release-ready Northstar Commerce API

```
REQUEST
  lab: 12
  objective: Produce a clean, testable release bundle with API
             docs, frontend assets and operational
             instructions.

EXPECTED EVIDENCE
  artifact: release-ready Northstar Commerce API
  codes: A4, A6, A8
  status: verified
```

ASSESSMENT BRIDGE

Ability codes
A4, A6, A8

Capture a screenshot, terminal output or test result that proves the acceptance criterion.

Acceptance and diagnosis

PASS WHEN

A clean copy installs, tests, starts, serves the storefront/CRM dashboard, completes catalog/customer/order flows and contains no secret or database file.

DIAGNOSE

1. Inspect HTTP status and response body
2. Check request fields and headers
3. Follow request ID into logs
4. Re-run the smallest failing test

Prompt output must preserve the API boundary

TARGET FILES

`README.md`

`.env.example`

`release-checklist.md`

ACCEPTANCE

A clean copy installs, tests, starts, serves the storefront/CRM dashboard, completes catalog/customer/order flows and contains no secret or database file.

ABILITY TRACE

A4, A6, A8

WRAP-UP

Course Summary & Assessment

Evidence first: contract, code, tests, documentation

LEARNING OUTCOMES

What You Can Now Build

1

LO1

Identify and assess FastAPI as middleware for creating connections between web clients, applications and data stores.

2

LO2

Integrate data and functions into a browser web app through typed FastAPI routes and reusable Python services.

3

LO3

Support API-level integration using REST semantics, JSON and an OpenAPI 3.1 contract.

4

LO4

Perform tests and checks on FastAPI-to-SQLite connections using Swagger UI, pytest, logs and VS Code debugging.

5

LO5

Modify a FastAPI service to improve integration, validation, compatibility, performance and security.

ASSESSMENT

Final Assessment

1

WA (SAQ)

10 open-ended questions · K1-K5 · 60 minutes

2

Practical Performance

6 FastAPI/OpenAPI/SQLite tasks · A1-A8 · 90 minutes

3

Open book

Slides, Learner Guide and approved lab files

4

Submission

Upload to lms-tms.tertiaryinfotech.com

WA → PP → SIGN-OFF

Assessment Flow

1. Read briefing and take assessment attendance
2. Complete WA (SAQ)
3. Complete Practical Performance
4. Upload question papers and project bundle
5. Review decision and sign record

MANDATORY CLOSING

Digital Attendance and TRAQOM

1

Assessment attendance

Scan the SSG digital attendance QR code.

2

TRAQOM survey

Complete the course survey before leaving.

3

Competency

Complete all prescribed assessments.

4

Appeal

Follow the documented appeal route if you disagree with a decision.

BUILD · VERIFY · EXPLAIN

Thank You!

Your FastAPI service is only complete when its contract, tests, evidence and client agree.